

UNIVERSIDADE ESTADUAL PAULISTA "JÚLIO DE MESQUITA FILHO"
FACULDADE DE CIÊNCIAS - CAMPUS BAURU
DEPARTAMENTO DE COMPUTAÇÃO
BACHARELADO EM CIÊNCIA DA COMPUTAÇÃO

DANIEL GOMES CANDIDO

**EXPLORANDO ABORDAGENS SEM NORMALIZAÇÃO PARA
INFERÊNCIA EFICIENTE EM GRANDES MODELOS DE
LINGUAGEM**

BAURU
Novembro/2025

DANIEL GOMES CANDIDO

**EXPLORANDO ABORDAGENS SEM NORMALIZAÇÃO PARA
INFERÊNCIA EFICIENTE EM GRANDES MODELOS DE
LINGUAGEM**

Monografia do Trabalho de Conclusão de Curso
apresentada ao Curso de Bacharelado em Ciência da Computação da Universidade Estadual Paulista “Júlio de Mesquita Filho”, Faculdade de Ciências, Campus Bauru.

Orientador: Prof. Dr. Leandro Aparecido Passos Junior

Coorientador: Prof. Me. Pedro Henrique Paiola

BAURU

Novembro/2025

C217e	Candido, Daniel Gomes Explorando abordagens sem normalização para inferência eficiente em grandes modelos de linguagem / Daniel Gomes Candido. -- Bauru, 2025 62 p. Trabalho de conclusão de curso (Bacharelado - Ciência da Computação) - Universidade Estadual Paulista (UNESP), Faculdade de Ciências, Bauru Orientador: Leandro Aparecido Passos Junior Coorientador: Pedro Henrique Paiola 1. Ciência da computação. 2. Inteligência artificial. 3. Aprendizado do computador. 4. Processamento de linguagem natural (Computação). 5. Aprendizagem profunda (Aprendizado do computador). I. Título.
-------	--

Daniel Gomes Candido

Explorando Abordagens Sem Normalização para Inferência Eficiente em Grandes Modelos de Linguagem

Monografia do Trabalho de Conclusão de Curso apresentada ao Curso de Bacharelado em Ciência da Computação da Universidade Estadual Paulista “Júlio de Mesquita Filho”, Faculdade de Ciências, Campus Bauru.

Banca Examinadora

Prof. Dr. Leandro Aparecido Passos Junior
1º Membro - Orientador
Universidade Estadual Paulista "Júlio de Mesquita Filho"
Faculdade de Ciências
Departamento de Ciência da Computação

Dra. Simone das G. Domingues Prado
2º Membro - Profª Disciplina
Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP)
Faculdade de Ciências (FC) – Câmpus de Bauru
Departamento de Computação

Dr. Douglas Rodrigues
3º Membro - Convidado
Universidade Estadual Paulista "Júlio de Mesquita Filho" (UNESP)
Faculdade de Ciências (FC) – Câmpus de Bauru
Departamento de Computação

Bauru, 11 de novembro de 2025.

*Dedico este trabalho aos meus pais e irmãos, pelo amor e apoio constantes;
aos professores, pela paciência e pelo conhecimento compartilhado;
aos colegas e amigos, pela companhia nesta caminhada;
e, sobretudo, a Deus, em quem encontrei força, direção e sentido em cada etapa desta
jornada.*

Agradecimentos

Em primeiro lugar, agradeço a Deus, em quem encontrei força, direção e sentido em cada etapa desta jornada. Sem Ele, o vislumbre dos objetivos conquistados e a energia para atingi-los não teriam sido possíveis; sem Ele, as situações difíceis teriam se tornado insuportáveis e as partes boas, sem sentido.

Agradeço profundamente aos meus pais, pelo amor incondicional, pela educação, pelo exemplo de trabalho e, sobretudo, pelo apoio constante, tanto emocional quanto material, sem o qual nenhuma palavra desta monografia teria sido escrita. Aos meus irmãos, que sempre estiveram comigo, presencialmente ou por meio de palavras e pensamentos, na torcida de que tudo desse certo, sou igualmente grato pelo incentivo e pela companhia ao longo de toda a graduação.

Estendo meus agradecimentos a todos os professores que exerceram a nobre tarefa de compartilhar sua sabedoria e décadas de estudo comigo. Serei eternamente grato por toda a bagagem transmitida ao longo do curso, bem como pela paciência e compreensão em momentos cruciais, que me ajudaram a transpor diversas limitações pessoais. Em especial, agradeço ao meu orientador, Prof. Dr. Leandro Aparecido Passos Junior, e ao meu coorientador, Prof. Me. Pedro Henrique Paiola, pela orientação dedicada, pelas sugestões valiosas e pela confiança depositada neste trabalho.

Agradeço também aos colegas e amigos que compartilharam comigo tantos momentos excelentes, remota ou presencialmente. Com eles aprendi muito, seja em discussões técnicas, seja em conversas informais que tornaram a caminhada mais leve. Espero, sinceramente, ter contribuído de alguma forma para o crescimento de cada um, deixando algo que possa ser útil em algum momento de suas trajetórias.

Por fim, agradeço à Universidade Estadual Paulista “Júlio de Mesquita Filho” e à Faculdade de Ciências, pelo ambiente de aprendizado, pelas oportunidades oferecidas e por todo o suporte institucional que possibilitou a realização deste trabalho.

*“Bem-aventurado o que espera e chega até mil trezentos e trinta e cinco dias.
Tu, porém, vai até ao fim; porque repousarás e estarás na tua sorte, no fim dos dias.”*
Daniel 12:12–13

Resumo

Este trabalho apresenta estudo experimental comparativo que avalia uma simplificação arquitetural no GPT-2 - remoção de *LayerNorm* (*LN-Free*) - combinada a métodos de adaptação parametricamente eficiente (*PEFT*) para acelerar a inferência preservando a qualidade. Adota o GPT-2 *small* oficial como *baseline* e uma variante *LN-Free* pública; executa experimentos no *wikitext-2-raw-v1* (blocos de 1024 tokens) medindo *perplexity* (PPL) e vazão em tokens por segundo. O *baseline* oficial alcança $\text{PPL} = 30,88$; a variante *LN-Free* sem ajuste obtém $\text{PPL} = 36,54$ e aumento de *throughput* de cerca de 23,5% em $\text{seq} = 896$ e $\text{batch} = 1$, com ganhos consistentes em outras configurações. Sob um orçamento de treino unificado (300 passos), LoRA ($r = 8$) atinge a menor PPL (26,64) no modelo *vanilla* e 28,46 na variante LN-Free (0,94% de parâmetros treináveis); BitFit obtém $\text{PPL} = 28,18$ (no *vanilla*) e 30,48 (no LN-Free) com 0,082% de parâmetros; IA³ alcança $\text{PPL} = 30,29$ (no *vanilla*) com 0,044% de parâmetros; e AdaLoRA, sob o mesmo regime de treino, diverge ($\text{PPL} > 50,0$), mostrando-se inadequado para este orçamento curto. Conclui-se que a remoção de normalizações reduz o custo da inferência, e que *PEFT* (especialmente LoRA e BitFit) recupera e supera a qualidade do *baseline*. A combinação LN-Free + LoRA ($\text{PPL} 28,46$) apresenta o melhor balanço, sendo mais rápida e mais precisa que o *baseline* original ($\text{PPL} 30,88$). Discute limitações (horizonte de treino e variabilidade de *runtime*) e propõe extensões (agendamento mais longo, latência p95 e generalização a outros modelos e tarefas).

Palavras-chave: Grandes Modelos de Linguagem; Inferência; LayerNorm; GPT-2; PEFT; LoRA; IA³; AdaLoRA; BitFit; Desempenho.

Abstract

This paper reports a comparative experimental study that evaluates an architectural simplification in GPT-2 - removing LayerNorm (LN-Free) - combined with parameter-efficient fine-tuning (PEFT) methods to accelerate inference while preserving quality. It adopts the official GPT-2 small as the baseline and a public LN-Free variant; experiments on *wikitext-2-raw-v1* (1024-token blocks) measure perplexity (PPL) and token-per-second throughput. The official baseline reaches $\text{PPL} = 30.88$; the unadjusted LN-Free variant yields $\text{PPL} = 36.54$ and about 23.5% higher throughput at $\text{seq} = 896$ and $\text{batch} = 1$, with consistent gains elsewhere. Under a unified training budget (300 steps), LoRA ($r = 8$) achieves the best PPL (26.64) on the vanilla model and 28.46 on the LN-Free variant (0.94% trainable parameters); BitFit attains $\text{PPL} = 28.18$ (vanilla) and 30.48 (LN-Free) with 0.082% parameters; IA³ reaches $\text{PPL} = 30.29$ (vanilla) with 0.044% parameters; and AdaLoRA, under the same regime, diverged ($\text{PPL} > 50.0$), proving unsuitable for this short budget. The study concludes that removing normalizations reduces inference cost, while PEFT (notably LoRA and BitFit) recovers and surpasses baseline quality. The LN-Free + LoRA combination ($\text{PPL} 28.46$) offers the best balance, being both faster and more accurate than the original baseline ($\text{PPL} 30.88$). It discusses limitations (short training horizon and runtime variability) and outlines extensions (longer scheduling, p95 latency, and generalization to other models and tasks).

Keywords: Large Language Models; Inference; LayerNorm; GPT-2; PEFT; LoRA; IA³; AdaLoRA; BitFit; Performance.

Lista de Figuras

Figura 1 – Visão geral da arquitetura Transformer encoder–decoder.	19
Figura 2 – Detalhe do bloco Multi-Head Attention.	21
Figura 3 – Cálculo da atenção escalonada (<i>Scaled Dot-Product</i>).	22

Lista de Tabelas

Tabela 1	–	Versões de bibliotecas recomendadas para reprodutibilidade.	46
Tabela 2	–	Resultados dos Baselines (A1 e A2) — avaliação de PPL e inferência (<i>single-batch</i> , 512 tokens) no dataset <i>Wikitext</i> completo.	51
Tabela 3	–	Resultados do <i>fine-tuning</i> PEFT (A3–A6) após 300 passos de treino no dataset <i>Wikitext</i> (tamanho de lote efetivo = 32, taxa de aprendizado = 2e-4).	52
Tabela 4	–	<i>Benchmark</i> de <i>throughput</i> de inferência (tokens/s) dos modelos <i>baseline</i> , variando o tamanho da sequência (Seq) e o tamanho do lote (Batch). .	53

Lista de abreviaturas e siglas

AdaLoRA	<i>Adaptive Low-Rank Adaptation</i>
API	<i>Application Programming Interface</i>
BERT	<i>Bidirectional Encoder Representations from Transformers</i>
BitFit	<i>Bias Fine-Tuning</i>
CNN	<i>Convolutional Neural Network</i>
CPU	Unidade Central de Processamento
FFN	<i>Feed-Forward Network</i>
FP16	<i>Half-Precision Floating Point</i>
FP32	<i>Single-Precision Floating Point</i>
GPT	<i>Generative Pre-trained Transformer</i>
GPU	Unidade de Processamento Gráfico
IA ³	<i>Infused Adapter by Inhibiting and Amplifying Inner Activations</i>
JSON	<i>JavaScript Object Notation</i>
LLM	<i>Large Language Model</i>
LN	<i>Layer Normalization</i>
LN-Free	Arquitetura sem <i>LayerNorm</i>
LoRA	<i>Low-Rank Adaptation</i>
MHA	<i>Multi-Head Attention</i>
PEFT	<i>Parameter-Efficient Fine-Tuning</i>
PLN	Processamento de Linguagem Natural
PPL	<i>Perplexity</i>
QKV	<i>Query, Key, Value</i>
RNN	<i>Recurrent Neural Network</i>

Sumário

1	INTRODUÇÃO	14
1.1	Contexto e motivação	14
1.2	Problema e escopo	14
1.3	Objetivos	15
1.4	Hipótese e questões de pesquisa	15
1.5	Resumo dos experimentos e resultados utilizados	16
1.6	Contribuições	16
1.7	Relevância, contextualização e impacto	17
1.8	Organização do trabalho	17
2	REVISÃO BIBLIOGRÁFICA	18
2.1	Aspectos Históricos e Fundamentação dos Transformers	18
2.2	Funcionamento da Arquitetura Transformer	19
2.2.1	Positional Encoding	20
2.2.2	Atenção Multicabeça (<i>Multi-Head Attention</i>)	20
2.2.3	Atenção escalonada (<i>Scaled Dot-Product</i>)	21
2.2.4	Bloco <i>Encoder/Decoder</i>	23
2.3	Grandes Modelos de Linguagem e Processamento de Linguagem Natural	24
2.4	Trabalhos Correlatos	24
2.4.1	Normalização em <i>Transformers</i> e alternativas	24
2.4.2	Remoção ou adaptação de normalizações em modelos pré-treinados	25
2.4.3	Adaptação parametricamente eficiente (<i>PEFT</i>)	26
2.4.4	Aceleração da inferência em modelos de linguagem	28
3	METODOLOGIA	29
3.1	Visão geral do delineamento	29
3.2	Modelos e <i>checkpoints</i>	30
3.3	<i>Tokenizer</i> e pré-processamento	31
3.4	Dados	32
3.5	Métricas	33
3.5.1	Qualidade preditiva: perplexidade	33
3.5.2	Eficiência de inferência: <i>throughput</i> e uso de memória	33
3.6	Ambiente e controle experimental	34
3.7	Protocolo de avaliação de perplexidade	36
3.8	Benchmark de inferência	37

3.9	Métodos de adaptação avaliados	38
3.9.1	<i>LoRA</i>	39
3.9.2	<i>BitFit</i>	39
3.9.3	<i>AdaLoRA</i>	40
3.9.4	<i>IA³</i>	40
3.10	Hiperparâmetros de treino	41
3.11	Tratamento da variante <i>LN-Free</i>	42
3.12	<i>Sweep</i> de inferência	44
3.13	Armazenamento e organização de resultados	44
3.14	Instruções de reprodutibilidade	45
3.15	Script <i>Python</i> resumido para reprodutibilidade	47
3.16	Metodologia detalhada de avaliação	49
3.17	Limitações metodológicas	50
4	RESULTADOS	51
4.1	Resumo dos experimentos	51
4.2	<i>Sweep</i> de inferência	53
4.3	Análise dos resultados	53
5	DISCUSSÃO	55
5.1	Visão geral e aderência à hipótese	55
5.2	Qualidade preditiva: efeitos e recuperação	55
5.3	Eficiência de inferência e perfil de ganhos	56
5.4	Custo/benefício paramétrico nos métodos de adaptação	56
5.5	Interpretação prática e escolhas de projeto	57
5.6	Ameaças à validade e limitações	58
5.7	Implicações e recomendações	58
6	CONCLUSÃO	59
	REFERÊNCIAS BIBLIOGRÁFICAS	62

1 Introdução

1.1 Contexto e motivação

A popularização dos *Large Language Models* (LLMs) baseados em *Transformer* trouxe avanços expressivos em tarefas de Processamento de Linguagem Natural (PLN), mas também elevou o custo operacional durante a etapa de inferência, especialmente em cenários de borda e tempo real (RAIAAN *et al.*, 2024; SHAO *et al.*, 2024; ZHANG *et al.*, 2025). Em aplicações interativas, embarcadas ou com restrições de custo, a latência por token e o consumo de memória tornam-se gargalos frequentes. Nesse cenário, estratégias arquiteturais e de adaptação *Parameter-Efficient Fine-Tuning* (PEFT) têm ganhado destaque por permitirem ganhos de eficiência com mínima degradação de qualidade (SHAO *et al.*, 2024).

Entre as linhas de investigação, duas se mostram particularmente promissoras e complementares: a simplificação arquitetural por remoção de normalizações, como a *Layer Normalization* (LN), e o uso de técnicas de *fine-tuning* parametricamente eficiente, tais como *Low-Rank Adaptation* (LoRA), *BitFit*, *AdaLoRA* e (*Infused Adapter by Inhibiting and Amplifying Inner Activations*) (IA³). A primeira atua diretamente na etapa *forward* da geração de tokens, reduzindo operações ponto a ponto e movimentos de memória na pilha de normalização, o que tende a beneficiar a inferência em sequências longas, especialmente em cargas interativas. A segunda, por sua vez, busca recuperar ou até melhorar a qualidade após uma alteração estrutural, treinando apenas uma fração dos parâmetros. Evidências recentes mostram que é possível remover camadas de LN em GPT-2 *small* via *fine-tuning* mantendo desempenho próximo ao original, o que motiva a presente investigação (HEIMERSHEIM, 2024).

Contudo, a simples remoção de normalização tende a degradar a qualidade se não houver compensações adequadas. Persistem lacunas quanto a: (i) quantificar o impacto da remoção de *LayerNorm* em diferentes regimes de comprimento de sequência e tamanhos de *batch*; (ii) combinar simplificação arquitetural com adaptação parametricamente eficiente (por exemplo, *LoRA*, *BitFit*, *IA*³) para recuperar qualidade; e (iii) caracterizar o balanço qualidade–desempenho–uso de memória sob condições realistas de inferência.

1.2 Problema e escopo

O problema central deste trabalho é o alto custo computacional da etapa de inferência em Grandes Modelos de Linguagem (LLMs), tanto em tempo de resposta quanto

em uso de memória, o que dificulta sua adoção em cenários com recursos limitados. Diante disso, este trabalho investiga se a remoção da normalização, associada a métodos de adaptação leve, em um modelo *Transformer* pré-treinado de porte reduzido (GPT-2 *small*) pode trazer benefícios concretos de desempenho na inferência e de que forma esses métodos compensam a perda de qualidade decorrente dessa simplificação.

O estudo foca em tomar o alto custo computacional da inferência em LLMs como problema de partida, utilizando o GPT-2 *small* como modelo de referência, comparar um baseline oficial do GPT-2 *small* com uma variante sem LN (LN-Free), quantificar os efeitos em tempo de inferência e uso de memória para diferentes comprimentos de sequência e tamanhos de *batch*, e avaliar em que medida a qualidade pode ser recuperada por meio de LoRA, *BitFit*, *AdaLoRA* e IA³, sob orçamentos de treino curtos.

1.3 Objetivos

Objetivo geral

Investigar a eficiência de inferência e a qualidade preditiva de um GPT-2 *small* sem LN, bem como o impacto de técnicas de adaptação PEFT na recuperação de qualidade.

Objetivos específicos

1. Estabelecer um baseline oficial com GPT-2 *small* e uma variante LN-Free, medindo *Perplexity* (PPL) e *throughput* de inferência.
2. Mensurar ganhos de *throughput* do LN-Free frente ao baseline para diferentes comprimentos de sequência e tamanhos de *batch*.
3. Aplicar LoRA, *BitFit*, *AdaLoRA* e IA³ à variante LN-Free, quantificando a recuperação de qualidade com poucos passos de treino.
4. Relacionar a fração de parâmetros treináveis de cada método com a PPL final e o custo de inferência observado.

1.4 Hipótese e questões de pesquisa

A hipótese central é que a remoção da LN reduz o custo de inferência, sobretudo em sequências longas e em *batch* reduzido, e que métodos de adaptação leve permitem recuperar uma parcela significativa da qualidade com baixo custo de treino (HEIMERSHEIM, 2024; ZHANG *et al.*, 2025).

A partir disso, derivam-se as seguintes questões: em que faixas de sequência e *batch* os ganhos de *throughput* do LN-Free são mais evidentes? Qual método de adaptação leve apresenta melhor compromisso entre qualidade (PPL), fração de parâmetros treináveis e custo de inferência? Orçamentos curtos de treino são suficientes para estabilizar *AdaLoRA* e IA^3 , ou demandam mais passos e ajustes de *scheduler*?

1.5 Resumo dos experimentos e resultados utilizados

Os experimentos foram conduzidos com GPT-2 *small* oficial como *baseline* e uma variante LN-Free disponibilizada publicamente (HEIMERSHEIM, 2024). Utilizou-se o conjunto *wikitext-2-raw-v1* com *block size* de 1024, e medidas padronizadas de PPL e *throughput* de inferência, sob um regime de treino unificado (300 passos, LR 2e-4, *batch* efetivo 32) para garantir a comparabilidade. Os principais resultados consolidados que fundamentam a monografia são:

- *Baseline* GPT-2 *small* (A1): PPL de aproximadamente 30,88.
- LN-Free (A2, sem adaptação): PPL de aproximadamente 36,54.
- *Sweep* de inferência: Ganhos de *throughput* para LN-Free, alcançando +23,5% (104,47 vs. 84,56 toks/s) em sequência longa (*seq*=896) e *batch* unitário.
- LoRA (A3, $r=8$): Atingiu a melhor PPL no *baseline* (26,64) e a melhor PPL no LN-Free (28,46), com 0,94% de parâmetros treináveis.
- *BitFit* (A4): Apresentou o segundo melhor resultado (PPL 28,18 no *baseline*; 30,48 no LN-Free) com apenas 0,082% de parâmetros.
- $(IA)^3$ (A6): Mostrou impacto modesto (PPL 30,29 no *baseline*; 34,54 no LN-Free) com 0,044% de parâmetros.
- *AdaLoRA* (A5): Divergiu sob o regime de treino unificado (PPL > 50,0), mostrando-se inadequado para este orçamento curto.

1.6 Contribuições

As contribuições deste trabalho são três. Primeiro, apresenta-se evidência experimental de que a remoção de normalização em GPT-2 *small* pode acelerar a inferência, especialmente em entradas longas e com *batch* pequeno. Em segundo lugar, realiza-se uma avaliação sistemática de LoRA, *BitFit*, *AdaLoRA* e IA^3 na recuperação de qualidade sobre a variante LN-Free, incluindo a análise das frações de parâmetros treináveis e medições de *throughput*. Por fim, disponibiliza-se um conjunto de resultados reprodutíveis, bem como

os notebooks e scripts utilizados nos experimentos, organizados em repositório público (CANDIDO, 2025), que podem servir de referência para estudos subsequentes de eficiência em inferência.

1.7 Relevância, contextualização e impacto

A análise sistemática de variantes sem normalização, aliada a métodos de adaptação leve, endereça demandas práticas de implantação de LLMs: menor latência, maior vazão e redução de consumo energético — com custo de ajuste modesto e sem retrabalho de pré-treinamento. Tais resultados são relevantes para aplicações interativas e para a sustentabilidade de infraestruturas de IA em larga escala, oferecendo um caminho incremental e economicamente viável para melhorar a eficiência de inferência.

1.8 Organização do trabalho

O Capítulo 1 apresenta o contexto, os objetivos, a hipótese e um resumo dos resultados. O Capítulo 2 apresenta a revisão bibliográfica sobre *Transformer*, LN e métodos de adaptação leve (*PEFT*). Em seguida, o Capítulo 3 descreve a metodologia, detalhando modelos, dados, procedimentos e métricas adotadas. O Capítulo 4 reúne os resultados experimentais e as análises comparativas. O Capítulo 5 discute os achados à luz da hipótese e da literatura, apontando limitações e implicações práticas. Por fim, o Capítulo 6 traz a síntese conclusiva e propõe trabalhos futuros.

2 Revisão Bibliográfica

Os LLMs revolucionaram o campo do PLN, permitindo avanços em tarefas como tradução automática, geração de texto e assistentes virtuais. No entanto, a eficiência computacional durante a inferência — etapa em que o modelo gera previsões com base em novos dados — permanece um desafio crítico, principalmente devido ao alto custo das camadas de normalização como a *LayerNorm* (CHITTY-VENKATA *et al.*, 2023; SONG *et al.*, 2024). Este capítulo apresenta uma visão integrada dos aspectos históricos, fundamentos teóricos, principais técnicas de otimização de inferência e avanços recentes em abordagens sem normalização.

2.1 Aspectos Históricos e Fundamentação dos Transformers

O desenvolvimento dos LLMs pode ser rastreado até os primeiros modelos estatísticos de linguagem, mas o grande avanço ocorreu com a introdução da arquitetura *Transformer* por Vaswani *et al.* (2017). Diferentemente das redes neurais recorrentes (RNNs) e convolucionais (CNNs), os Transformers baseiam-se exclusivamente em mecanismos de autoatenção (*self-attention*), permitindo que o modelo processe sequências de maneira paralela e capture dependências de longa distância, tornando-se referência em tarefas como tradução e sumarização.

A arquitetura Transformer é composta por codificadores (*encoders*) e decodificadores (*decoders*), cada um contendo blocos de autoatenção multi-cabeça, redes *feed-forward* e camadas de normalização como a *LayerNorm*. Essas camadas estabilizam o treinamento, mitigando problemas como o desaparecimento de gradientes, mas introduzem custos computacionais relevantes na inferência de modelos de larga escala (VASWANI *et al.*, 2017; SONG *et al.*, 2024).

2.2 Funcionamento da Arquitetura Transformer

O Transformer, proposto por Vaswani *et al.* (2017), baseia-se em três pilares principais: *self-attention* escalonada, codificador–decodificador empilhados e processamento totalmente paralelo (sem recorrência). A Figura 1 ilustra os blocos essenciais do modelo Transformer, cujas partes principais são detalhadas nas subseções seguintes.

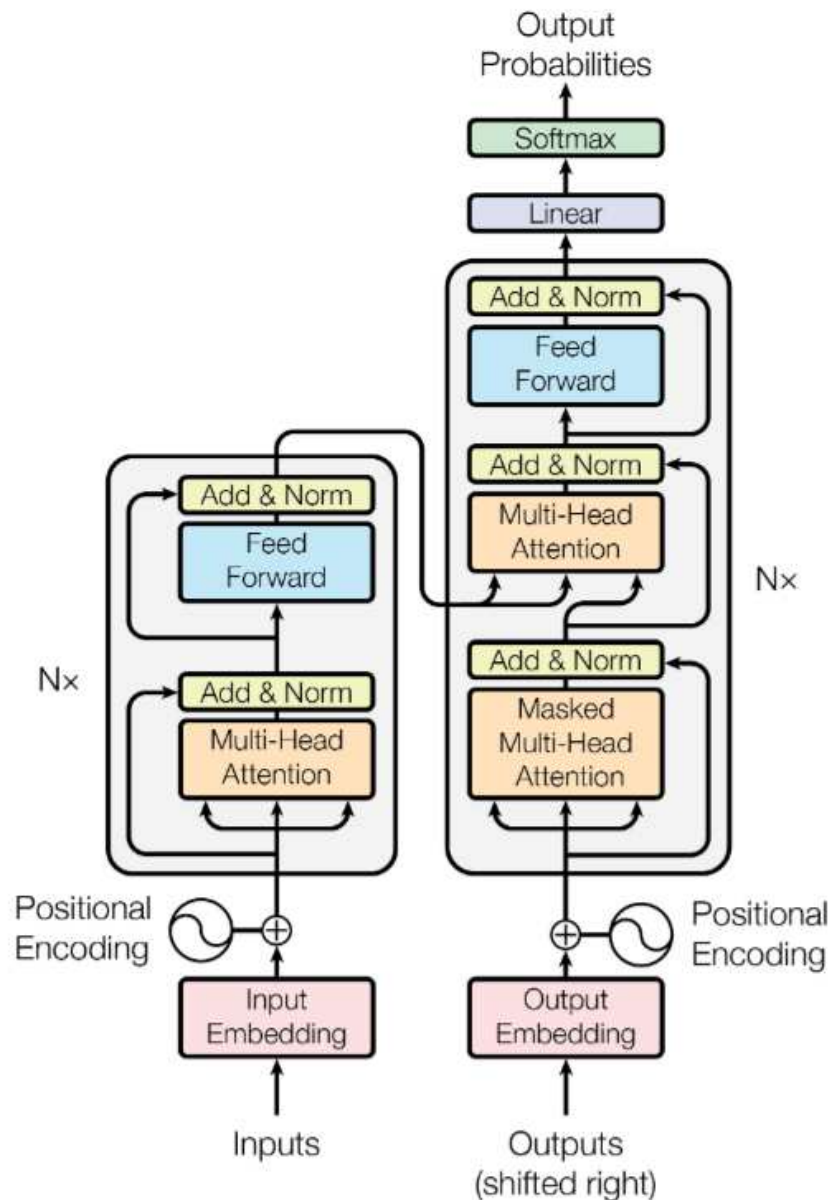


Figura 1 – Visão geral da arquitetura Transformer encoder–decoder.

Fonte: Adaptado de Vaswani *et al.* (2017).

2.2.1 Positional Encoding

Como o modelo *Transformer* dispensa mecanismos recorrentes e convolucionais, ele não possui, por construção, noção explícita da ordem dos tokens na sequência de entrada. Para que a rede consiga distinguir, por exemplo, se uma palavra aparece no início ou no fim da frase, é necessário injetar informação de posição em cada vetor de *embedding*.

Para isso, associa-se a cada posição i de uma sequência um vetor de posição $\mathbf{p}_i \in \mathbb{R}^d$, com a mesma dimensionalidade d do *embedding* da palavra $\mathbf{e}_i$. O vetor de entrada efetivo em cada passo é então dado por

$$\mathbf{x}_i = \mathbf{e}_i + \mathbf{p}_i,$$

de modo que o modelo receba simultaneamente informação de conteúdo (semântica) e de posição.

No *Transformer* original, esses vetores $\mathbf{p}_i$ são determinísticos e calculados por funções senoidais de diferentes frequências, o que permite ao modelo inferir relações relativas entre posições (como distância entre tokens) sem armazenar explicitamente um índice inteiro. Alternativamente, trabalhos posteriores exploram codificações posicionais aprendidas, nas quais $\mathbf{p}_i$ é parametrizado e ajustado durante o treinamento, preservando a ideia central de que cada token carrega, junto ao seu *embedding* lexical, uma representação contínua da sua posição na sequência.

2.2.2 Atenção Multicabeça (*Multi-Head Attention*)

Na autoatenção padrão, cada posição da sequência gera vetores de consulta, chave e valor (Q , K e V) a partir das representações de entrada, permitindo que o modelo combine informações de diferentes tokens. A atenção multicabeça estende esse mecanismo ao replicar a operação de atenção h vezes em sub-espacos de menor dimensão, chamados de *cabeças* de atenção.

Em termos formais, para cada cabeça $j \in \{1, \dots, h\}$ são aprendidas projeções lineares específicas W_j^Q , W_j^K e W_j^V , que geram consultas, chaves e valores próprios daquele sub-espaco. A saída da cabeça j é dada por

$$\text{head}_j = \text{Attention}(QW_j^Q, KW_j^K, VW_j^V), \quad (2.1)$$

em que $\text{Attention}(\cdot)$ representa o mecanismo de atenção escalonada descrito anteriormente. Cada cabeça opera, portanto, sobre uma projeção diferente dos mesmos vetores de entrada, permitindo capturar padrões distintos de dependência (por exemplo, relações sintáticas, semânticas locais ou globais).

As saídas das h cabeças são então concatenadas ao longo da dimensão de características e projetadas novamente para o espaco original de dimensão d_{model} :

$$\text{MHA}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O, \quad (2.2)$$

em que W^O é uma matriz de projeção aprendida. Dessa forma, o modelo aumenta a expressividade sem incrementar excessivamente o custo computacional, já que cada cabeça opera em um sub-espço de dimensão reduzida (tipicamente d_{model}/h). Na prática, isso permite que o Transformer aprenda, em paralelo, múltiplas maneiras de relacionar os tokens de uma mesma sequência.

A Figura 2 ilustra o bloco de atenção multicabeça, evidenciando a presença de várias cabeças de autoatenção seguidas pelas operações de concatenação, projeção de saída e normalização residual.

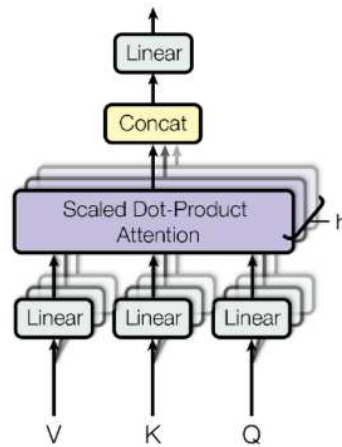


Figura 2 – Detalhe do bloco Multi-Head Attention.

Fonte: Adaptado de Vaswani *et al.* (2017).

2.2.3 Atenção escalonada (*Scaled Dot-Product*)

O núcleo do mecanismo de atenção no *Transformer* é a atenção escalonada por produto interno. Para cada posição da sequência, o modelo associa um vetor de consulta $\mathbf{q}$ (*query*), um conjunto de vetores de chave $\mathbf{k}_i$ (*keys*) e vetores de valor $\mathbf{v}_i$ (*values*), todos em $\mathbb{R}^{d_k}$. A ideia é medir, por meio de produtos internos, o quanto a consulta $\mathbf{q}$ deve “prestar atenção” a cada chave $\mathbf{k}_i$ e, a partir desses escores, combinar os valores $\mathbf{v}_i$ em uma representação ponderada.

De forma matricial, considerando uma consulta $\mathbf{q}$, uma matriz de chaves K (cujas linhas são as chaves $\mathbf{k}_i$) e uma matriz de valores V (cujas linhas são os valores $\mathbf{v}_i$), a atenção escalonada é definida por

$$\text{Attention}(\mathbf{q}, K, V) = \text{softmax}\left(\frac{\mathbf{q}K^\top}{\sqrt{d_k}}\right) V, \quad (2.3)$$

em que d_k é a dimensionalidade dos vetores de chave e $\text{softmax}(\cdot)$ é aplicada componente a componente sobre o vetor de escores. O fator de escala $1/\sqrt{d_k}$ reduz a magnitude dos produtos internos quando d_k é grande, evitando que os valores de *softmax* se tornem

excessivamente concentrados em poucos elementos e contribuindo para a estabilidade numérica do treinamento.

Em termos intuitivos, o vetor de escores $\mathbf{q}K^\top$ mede a similaridade entre a consulta e cada chave da sequência. Após a normalização pela *softmax*, obtém-se um vetor de pesos $\boldsymbol{\alpha}$, com $\sum_i \alpha_i = 1$, que indica quanto cada posição da sequência deve influenciar a saída. A combinação ponderada desses pesos com os valores $\mathbf{v}_i$ produz uma nova representação contextualizada para o token associado à consulta $\mathbf{q}$, sintetizando informações de todas as demais posições relevantes da sequência.

Esse mecanismo pode ser estendido naturalmente para um conjunto de consultas empilhadas em uma matriz Q , permitindo o cálculo da atenção para todos os tokens de uma sequência em paralelo, o que é uma das principais vantagens computacionais do *Transformer*. A Figura 3 ilustra o fluxo de cálculo da atenção escalonada, desde os produtos internos entre consultas e chaves até a aplicação da *softmax* e a combinação final com os valores.

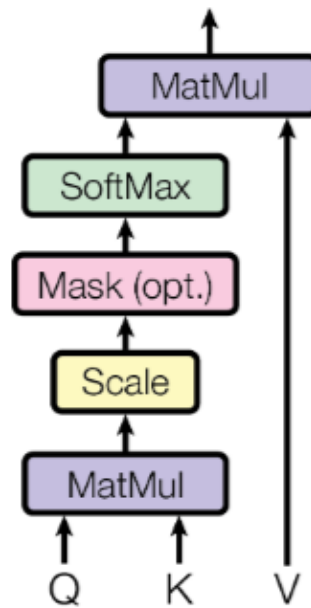


Figura 3 – Cálculo da atenção escalonada (*Scaled Dot-Product*).

Fonte: Adaptado de Vaswani *et al.* (2017).

Ao permitir que cada token “assista” a todos os demais por meio desses pesos de atenção, a atenção escalonada captura dependências de longa distância na sequência sem recorrer a mecanismos recorrentes, preservando paralelismo e eficiência na etapa de inferência.

2.2.4 Bloco *Encoder/Decoder*

A arquitetura do *Transformer* é organizada como um empilhamento de blocos idênticos de *encoder* e *decoder*. Cada *encoder* recebe uma sequência de vetores de entrada já enriquecidos com codificação posicional e produz representações contextualizadas, enquanto cada *decoder* consome essas representações do *encoder* para gerar, passo a passo, a sequência de saída.

Em um bloco de *encoder*, a camada é composta por três componentes principais:

- (i) uma camada de Atenção Multicabeça (*Multi-Head Attention*), que permite a cada posição da sequência considerar todas as demais posições por meio do mecanismo de atenção escalonada;
- (ii) uma rede *feed-forward* (*Feed-Forward Network* – FFN) aplicada de forma independente a cada posição, composta por duas projeções lineares separadas por uma ativação não linear (por exemplo, *ReLU* ou *GELU*);
- (iii) conexões residuais (*skip connections*) seguidas de normalização de camada (*Layer-Norm*) em cada subbloco.

Mais especificamente, dado um vetor de entrada $\mathbf{x}$ para a camada l , o subbloco de atenção produz:

$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{MHA}(\mathbf{x})), \quad (2.4)$$

em que $\text{MHA}(\cdot)$ denota a atenção multicabeça aplicada à sequência completa. Em seguida, o subbloco de rede *feed-forward* é definido por

$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z})), \quad (2.5)$$

onde $\text{FFN}(\cdot)$ consiste, tipicamente, em duas camadas lineares com uma ativação não linear entre elas. As conexões residuais preservam o gradiente e facilitam o treinamento de redes profundas, enquanto a *LayerNorm* estabiliza as distribuições internas de ativação.

No *decoder*, o bloco possui uma estrutura similar, mas com um componente adicional de atenção cruzada (*encoder-decoder attention*). Cada camada de *decoder* inclui:

- (i) uma Atenção Multicabeça *mascarada* sobre a própria sequência de saída parcial, garantindo a natureza autoregressiva da geração (cada posição só enxerga tokens anteriores);
- (ii) uma Atenção Multicabeça *encoder-decoder*, em que as consultas provêm das representações parciais do *decoder*, e chaves/valores provêm das saídas do *encoder*;
- (iii) uma rede *feed-forward* com duas projeções lineares e ativação não linear;

(iv) conexões residuais e *LayerNorm* em cada um dos três subblocos.

Assim, a saída intermediária do *decoder* é obtida encadeando os três subblocos com o mesmo padrão de *residual + LayerNorm*. A atenção mascarada garante que o modelo não utilize informações futuras ao prever o próximo token, enquanto a atenção encoder-decoder permite que cada posição na sequência de saída se apoie diretamente nas representações codificadas da entrada. Essa organização modular, comum a todas as camadas, torna o *Transformer* altamente paralelizável e flexível, ao mesmo tempo em que separa claramente o papel de codificar a entrada (blocos de *encoder*) e de gerar a saída autoregressiva (blocos de *decoder*).

2.3 Grandes Modelos de Linguagem e Processamento de Linguagem Natural

O PLN abrange o desenvolvimento de algoritmos capazes de compreender e gerar linguagem humana, com aplicações em assistentes virtuais, tradução e geração de conteúdo. LLMs como BERT, GPT e LLaMA, todos baseados em Transformers, impulsionaram resultados de ponta no PLN, sendo pré-treinados em grandes volumes de texto e ajustados para tarefas específicas. Entretanto, sua elevada demanda computacional durante a inferência limita a adoção em cenários de tempo real ou dispositivos restritos (LabelYourData, 2025; Intel Developer Zone, 2025).

2.4 Trabalhos Correlatos

Esta seção apresenta a revisão de literatura em quatro eixos relacionados ao tema desta monografia: (i) normalização em arquiteturas do tipo *Transformer* e alternativas; (ii) remoção ou adaptação de normalizações em modelos pré-treinados; (iii) métodos de adaptação parametricamente eficiente (*PEFT*); e (iv) técnicas gerais de aceleração da inferência em modelos de linguagem. A exposição está organizada em seções temáticas, com citações no sistema autor-data conforme as normas da ABNT.

2.4.1 Normalização em *Transformers* e alternativas

A normalização de camadas é elemento central em arquiteturas do tipo *Transformer*, tanto na variante *pre-norm* quanto na *post-norm* (VASWANI *et al.*, 2017). A *Layer Normalization* estabiliza distribuições internas de ativações, facilitando o treinamento de pilhas profundas com conexões residuais (BA; KIROS; HINTON, 2016). Entre alternativas, destaca-se a *RMSNorm*, que remove o centramento e normaliza pela média quadrática, reduzindo o número de operações (ZHANG; SENNRICH, 2019). Também se investigam

efeitos da posição da normalização no bloco (antes ou depois dos submódulos), com impacto sobre gradientes e estabilidade em profundidade (XIONG *et al.*, 2020).

2.4.2 Remoção ou adaptação de normalizações em modelos pré-treinados

Uma linha recente de pesquisa explora a simplificação do grafo de inferência após o pré-treinamento, por meio da redução ou remoção de camadas de normalização. No contexto do GPT-2, Heimersheim (HEIMERSHEIM, 2024) mostra que é possível remover gradualmente instâncias de *LayerNorm* em GPT-2 *small* e, com um ajuste fino direcionado sobre uma fração dos dados originais, recuperar desempenho próximo ao modelo de referência. Em geral, observa-se uma queda inicial de qualidade ao eliminar a normalização, seguida de recuperação parcial ou substantiva com um curto período de *fine-tuning*. Os ganhos de eficiência de inferência tendem a se evidenciar em sequências longas, quando o custo agregado de operações ponto a ponto e movimentação de memória na pilha de normalização se torna mais relevante para a latência.

Trabalhos subsequentes ampliam essa linha de investigação. Baroni et al. estendem a remoção de *LayerNorm* para toda a família GPT-2, incluindo variantes de maior porte, como GPT-2 XL, demonstrando que todas as camadas de normalização podem ser eliminadas com aumento marginal de perda em conjuntos de validação (BARONI *et al.*, 2025). Esses resultados reforçam a hipótese de que a normalização exerce papel mais crítico na estabilização do treinamento do que na etapa de inferência, desde que o modelo seja convenientemente reajustado após a modificação estrutural. Além disso, a remoção de *LayerNorm* simplifica a análise mecanicista ao reduzir não linearidades adicionais no caminho residual.

Paralelamente à remoção direta, surgem propostas de adaptação mais suave, baseadas em reparametrizações de normalização em modelos já pré-treinados. Jiang et al. introduzem variantes *Pre-RMSNorm* e *Pre-CRMSNorm*, estabelecendo a equivalência funcional entre Transformers *Pre-LN* e arquiteturas com normalização baseada em RMS, tanto em treinamento quanto em inferência (JIANG *et al.*, 2023). Ao remover o centramento explícito e reaproveitar redundâncias na representação, esses autores mostram que é possível converter modelos *Pre-LN* para versões *Pre-RMSNorm/Pre-CRMSNorm* com ganhos de eficiência típicos entre 1% e 10% em tempo de execução, sem perda significativa de qualidade.

Outras abordagens propõem esquemas híbridos de normalização que combinam *LayerNorm* e *RMSNorm* em diferentes submódulos do bloco Transformer. Estratégias como a *HybridNorm* exploram o uso de normalizações mais estáveis em subcamadas sensíveis (por exemplo, após a atenção) e variantes mais leves em demais componentes, buscando um compromisso entre estabilidade de treinamento e custo computacional (ZHUO *et al.*, 2025). Em geral, esses métodos preservam a compatibilidade com pesos pré-treinados e

exigem apenas ajustes moderados nas escalas residuais e nos parâmetros de normalização.

Em síntese, a literatura recente aponta que tanto a remoção quanto a reparametrização da normalização em modelos pré-treinados são estratégias viáveis sob certas condições, com evidências de ganhos concretos em eficiência de inferência. No entanto, tais estudos ainda são pouco explorados sob a ótica de cenários realistas de inferência, especialmente no que se refere ao balanço entre qualidade, *throughput* e uso de memória em diferentes comprimentos de sequência e tamanhos de *batch*. O presente trabalho insere-se nesse contexto ao investigar experimentalmente a remoção de *LayerNorm* em GPT-2 *small* combinada a métodos *PEFT*, com foco explícito em latência, vazão e consumo de memória em regimes de inferência próximos ao uso prático.

2.4.3 Adaptação parametricamente eficiente (*PEFT*)

Métodos *PEFT* (*Parameter-Efficient Fine-Tuning*) visam adaptar modelos grandes atualizando apenas uma fração dos parâmetros, em contraste com o *fine-tuning* completo. Ao congelar os pesos originais e introduzir apenas pequenos conjuntos de parâmetros treináveis, essas técnicas reduzem o custo de treino (tempo, memória e armazenamento) e, em muitos casos, permitem que o caminho de inferência permaneça inalterado após a fusão das atualizações, evitando aumento de latência (HU *et al.*, 2021; LIU, 2022). Além disso, como os parâmetros adicionais são pequenos, torna-se viável manter múltiplas especializações de um mesmo modelo base, trocando apenas os “adaptadores” para cada tarefa.

Entre as técnicas mais utilizadas, destacam-se:

- *LoRA*: a *Low-Rank Adaptation* introduz decomposições de baixo ranque sobre projeções lineares já existentes (por exemplo, nas matrizes de atenção e nas camadas totalmente conectadas) (HU *et al.*, 2021). Em vez de atualizar diretamente uma matriz densa $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, o método mantém W congelada e aprende um incremento de baixo ranque $BA^\top$, em que $B \in \mathbb{R}^{d_{\text{out}} \times r}$ e $A \in \mathbb{R}^{d_{\text{in}} \times r}$, com $r \ll \min(d_{\text{out}}, d_{\text{in}})$. Assim, o modelo adaptado passa a usar $W' = W + BA^\top$ durante o treino. Na inferência, esse incremento pode ser fundido em W ou aplicado como multiplicações adicionais de custo praticamente desprezível, já que o ranque é pequeno. Estudos mostram que a *LoRA* pode reduzir em várias ordens de magnitude o número de parâmetros treináveis e o uso de memória durante o treino, mantendo desempenho comparável ao *fine-tuning* completo em modelos como GPT-2 e GPT-3 (HU *et al.*, 2021).
- *BitFit*: proposta como uma estratégia de ajuste extremamente enxuta, a *BitFit* congela todos os pesos do modelo e atualiza apenas os termos de viés (*bias*) em algumas ou em todas as camadas (ZAKEN; RAVFOGEL; GOLDBERG, 2022). O

racional é que esses vieses controlam deslocamentos nas ativações e podem capturar boa parte da adaptação necessária com pouquíssimos parâmetros adicionais. Em experimentos com modelos BERT, a *BitFit* se mostrou competitiva com o *fine-tuning* completo em cenários de dados pequenos e médios, e comparável a outros métodos esparsos em conjuntos maiores, tornando-se uma linha de base forte para PEFT com custo quase mínimo em armazenamento e treino (ZAKEN; RAVFOGEL; GOLDBERG, 2022).

- *AdaLoRA*: partindo da ideia de incrementos de baixo ranque, a *AdaLoRA* (*Adaptive LoRA*) observa que métodos baseados em decomposição normalmente distribuem de forma uniforme o orçamento de ranque entre todas as matrizes adaptadas, ignorando que algumas camadas são mais importantes do que outras (ZHANG, 2023). Para mitigar isso, a técnica parametriza os incrementos de baixo ranque via decomposição em valores singulares e estima, ao longo do treino, a importância de cada matriz. Com base nesses escores, a *AdaLoRA* realoca dinamicamente o orçamento de ranque, preservando mais componentes singulares em camadas sensíveis e podando atualizações em camadas menos relevantes. Essa redistribuição adaptativa tende a melhorar o desempenho em regimes de orçamento apertado, à custa de um processo de treino um pouco mais longo e sofisticado (ZHANG, 2023).
- IA^3 : o método IA^3 (*Infused Adapter by Inhibiting and Amplifying Inner Activations*) adota um enfoque ainda mais minimalista: em vez de introduzir matrizes de baixo ranque, ele aprende vetores (ou tensores de pequena dimensão) que reescalonom multiplicativamente subespaços das ativações internas (LIU, 2022). Na prática, vetores de escalonamento são aplicados a componentes específicos, como chaves e valores na autoatenção ou a saída de camadas *feed-forward*, modulando seletivamente a amplitude das ativações por tarefa. Como os pesos do modelo base permanecem congelados, o número de parâmetros adicionais é extremamente reduzido (tipicamente da ordem de 0,01 % dos parâmetros originais), e o custo computacional extra na inferência resume-se a algumas operações de multiplicação elemento a elemento, sem alteração estrutural das matrizes principais (LIU, 2022).

A literatura indica que a *LoRA* costuma apresentar maior capacidade de recuperação de qualidade quando aplicada a projeções de atenção e camadas totalmente conectadas, sobretudo quando se dispõe de um orçamento moderado de parâmetros adicionais (HU *et al.*, 2021). A IA^3 , por sua vez, maximiza a eficiência paramétrica, oferecendo bom compromisso entre desempenho e custo quando a prioridade é minimizar drasticamente o número de parâmetros treináveis (LIU, 2022). A *BitFit* atua como linha de base de custo mínimo, adequada para cenários de baixo orçamento ou como comparação de referência em estudos empíricos (ZAKEN; RAVFOGEL; GOLDBERG, 2022). Já a *AdaLoRA* tende

a beneficiar-se de horizontes de treino um pouco maiores para estabilizar a alocação adaptativa de ranques, explorando melhor o orçamento disponível em problemas mais exigentes (ZHANG, 2023).

Em cenários de simplificação arquitetural, como a remoção de normalizações em Transformers pré-treinados, esses métodos *PEFT* funcionam como mecanismos de recuperação de desempenho: o modelo base é modificado para reduzir o custo de inferência (por exemplo, eliminando *LayerNorm*), e as técnicas de adaptação leve são então usadas para readequar os parâmetros, mitigando a degradação de qualidade sem comprometer os ganhos de eficiência obtidos no grafo de inferência.

2.4.4 Aceleração da inferência em modelos de linguagem

Paralelamente à simplificação arquitetural e à adaptação leve, há um conjunto amplo de técnicas voltadas especificamente à aceleração da etapa de inferência. Entre elas, destacam-se a *kernel fusion*, que combina múltiplas operações em um único *kernel* para reduzir acessos à memória e sobrecarga de agendamento, mecanismos de atenção eficiente (como a *FlashAttention*), que reescrevem o cálculo de atenção para minimizar leituras e escritas em memória de alta latência, e o gerenciamento parcimonioso de memória e caches, incluindo reutilização de estados intermediários (DAO *et al.*, 2022). Abordagens de execução híbrida CPU–GPU também têm sido estudadas para melhorar a relação custo–desempenho em cenários práticos, deslocando partes menos sensíveis do fluxo de inferência para a CPU (SONG *et al.*, 2024). Além disso, estratégias de *batching* dinâmico e de pré-busca de chaves/valores contribuem para elevar o *throughput* sob múltiplas requisições concorrentes.

Essas técnicas são em grande medida ortogonais ao escopo desta monografia: enquanto a remoção de normalizações simplifica o grafo do modelo e reduz o número de operações lógicas na etapa *forward*, as otimizações de implementação exploram características do *hardware* e da pilha de execução. Em conjunto, ambas as frentes podem compor soluções mais eficientes de ponta a ponta para implantação de modelos de linguagem em produção.

3 Metodologia

Este capítulo descreve, de forma reprodutível, os modelos, dados, procedimentos, métricas e configurações empregados nos experimentos. Todos os resultados reportados nas seções de resultados e discussão derivam, sem exceção, dos protocolos estabelecidos aqui.

3.1 Visão geral do delineamento

O delineamento experimental deste trabalho parte da comparação entre dois modelos da mesma família e porte: o GPT-2 *small* oficial (baseline, A1) e uma variante sem *LayerNorm* (*LN-Free*, A2). Ambos possuem arquitetura compatível em termos de número de camadas, cabeças de atenção e dimensionalidade interna, diferindo principalmente pela presença (A1) ou remoção (A2) das camadas de normalização ao longo do bloco *Transformer*. Essa configuração permite isolar, de forma controlada, o efeito da remoção de *LayerNorm* sobre qualidade e eficiência de inferência.

Sobre cada um desses modelos base, foram avaliados quatro métodos de adaptação parametricamente eficiente (*PEFT*): *LoRA* (A3), *BitFit* (A4), *AdaLoRA* (A5) e *IA*³ (A6). Em todos os casos, o modelo pré-treinado original é mantido congelado, e apenas um subconjunto reduzido de parâmetros adicionais é treinado, seguindo o mesmo *pipeline* de dados e configuração de treino. Dessa forma, para cada combinação de arquitetura (com ou sem normalização) e técnica *PEFT*, obtém-se uma variante adaptada que busca recuperar (ou eventualmente melhorar) a qualidade preditiva perdida pela remoção de normalizações, sem reintroduzir custos significativos na etapa de inferência.

O estudo contempla duas dimensões principais de análise:

1. **Qualidade preditiva**, medida por perplexidade em validação, a partir de um *loop* de avaliação unificado sobre o conjunto *wikitext-2-raw-v1*. A perplexidade é calculada como exponencial da *loss* média de validação e serve como métrica padrão para comparar a capacidade dos modelos de modelar a distribuição de linguagem sob o mesmo regime de dados.
2. **Eficiência de inferência**, medida por *tokens por segundo* e memória de vídeo máxima (*VRAM*) alocada durante a geração. A taxa de *tokens/s* é obtida a partir de *benchmarks* controlados de geração autoregressiva, enquanto o uso de memória é monitorado via funções de coleta de estatísticas da GPU. Essas métricas permitem

avaliar os impactos práticos da remoção de normalizações e das adaptações *PEFT* em cenários de inferência realista.

Adicionalmente, realizou-se um *sweep* de inferência nos modelos *baseline* (A1 e A2b) variando sistematicamente o comprimento de sequência e o tamanho de *batch*. Esse experimento mapeia o comportamento de *throughput* em diferentes regimes de carga (sequências curtas vs. longas, *batch* pequeno vs. maior), permitindo identificar faixas em que a variante *LN-Free* oferece maior vantagem relativa. Os resultados desse *sweep* complementam as métricas pontuais de inferência e auxiliam na caracterização do trade-off entre qualidade, velocidade e uso de memória ao longo do espaço de configurações de inferência.

3.2 Modelos e *checkpoints*

Foram utilizados dois modelos da família GPT-2 com o mesmo porte (aproximadamente 124 milhões de parâmetros) e arquitetura compatível em termos de número de camadas, cabeças de atenção e dimensionalidade interna:

- **Baseline:** GPT-2 *small* oficial, identificado em repositório público como `gpt2` (também referido como `openai-community/gpt2`). Trata-se do modelo original disponibilizado pela OpenAI e amplamente adotado como referência em estudos de interpretabilidade e adaptação leve.
- **Variante *LN-Free*:** `schaeff/gpt2-small_LNFree300`, disponibilizado na plataforma Hugging Face como parte de uma família de modelos GPT-2 sem *LayerNorm* (BARONI *et al.*, 2025). Essa variante é obtida a partir do GPT-2 *small* oficial, removendo-se as normalizações dos blocos Transformer e a normalização final por substituição por identidades no grafo de inferência, seguida de um curto *fine-tuning* (300 passos) em *OpenWebText*. O procedimento resulta em um modelo com desempenho de validação próximo ao original, porém com um caminho de inferência simplificado e sem camadas de *LayerNorm*, conforme relatado nos resultados de validação para a família completa de modelos LN-free (BARONI *et al.*, 2025; HEIMERSHEIM, 2024).

A escolha desses *checkpoints* permite isolar o efeito da remoção de normalização: a variante *LN-Free* mantém pesos e hiperparâmetros de base comparáveis ao modelo *vanilla*, diferenciando-se principalmente pela ausência das operações de normalização no caminho *forward* de inferência e pelas pequenas correções introduzidas durante o *fine-tuning* de remoção.

Para ambos os casos, garantiu-se compatibilidade de *tokenizer* e de configuração de preenchimento. Utilizou-se o mesmo *tokenizer* `GPT2TokenizerFast`, com ajuste explícito de `pad_token_id` para coincidir com o `eos_token_id`, de modo a evitar conflitos na preparação dos lotes de validação e geração. Esse alinhamento é necessário porque o GPT-2 original não define um token de preenchimento nativo, e a padronização do *pad* sobre o token de fim de sequência é uma prática comum em experimentos de modelagem de linguagem autoregressiva, assegurando que ambos os modelos sejam avaliados sob condições idênticas de tokenização e mascaramento durante a inferência.

3.3 *Tokenizer* e pré-processamento

Foi utilizado o *tokenizer* oficial do GPT-2, na variante de implementação rápida `GPT2TokenizerFast`, baseada na biblioteca `tokenizers` da Hugging Face. Esse *tokenizer* emprega um esquema de *Byte Pair Encoding* (BPE) treinado sobre *WebText*, com vocabulário de 50 257 tokens, e é o padrão para os modelos `gpt2` e derivados. A adoção do mesmo *tokenizer* para os dois modelos avaliados (baseline e LN-Free) garante compatibilidade direta entre as sequências de entrada e saída, evitando que diferenças de segmentação introduzam viés nas comparações de perplexidade e desempenho de inferência.

Como o GPT-2 não define originalmente um token de preenchimento dedicado, configurou-se o token de preenchimento (`pad_token`) para coincidir com o token de fim de sequência (`eos_token`), prática comum em modelos autoregressivos. Adicionalmente, o *padding* foi ajustado globalmente para o lado esquerdo (`padding_side = 'left'`), de forma a preservar o alinhamento temporal dos tokens mais recentes no final da sequência. Essa configuração é importante para o correto funcionamento das rotinas de geração em *batch*, especialmente quando se utiliza *KV caching* interno: ao manter os tokens válidos alinhados à direita, o modelo pode reutilizar de forma eficiente os estados de atenção entre chamadas sucessivas.

O pré-processamento textual consistiu em concatenar os exemplos brutos do conjunto *wikitext-2-raw-v1* em um fluxo contínuo de texto, seguido de segmentação em blocos fixos de tamanho 1024 tokens (*block size*). Após a tokenização, as colunas textuais originais foram descartadas, preservando apenas as sequências codificadas e seus rótulos de linguagem. O valor de 1024 tokens foi escolhido por duas razões principais: (i) alinhar-se ao contexto máximo padrão do GPT-2 *small*, evitando truncamentos excessivos; e (ii) representar um regime de comprimento de sequência suficientemente desafiador para evidenciar diferenças de custo de inferência entre o modelo com *LayerNorm* e a variante LN-Free. Essa estratégia de blocagem fixa permite ainda a utilização eficiente de `DataLoaders` em *PyTorch*, com *batches* homogêneos em comprimento e sem necessidade de *padding* adicional na fase de avaliação.

3.4 Dados

O conjunto de dados empregado foi o *wikitext-2-raw-v1*, na partição original completa de treino e validação, conforme disponibilizada pela biblioteca `datasets` da Hugging Face. Trata-se de um corpus de texto contínuo extraído de artigos da Wikipédia, com linguagem natural relativamente formal e vocabulário variado, amplamente utilizado como *benchmark* de modelagem de linguagem para avaliar perplexidade em modelos do porte do GPT-2 *small*. A escolha desse *dataset* se justifica por três fatores principais: (i) ampla adoção na literatura, o que facilita a comparação qualitativa com estudos prévios; (ii) tamanho intermediário, suficiente para evitar sobreajuste imediato em poucas épocas, mas ainda viável em termos de custo computacional; e (iii) presença de sequências de comprimento moderado a longo, favorecendo a análise de efeitos de eficiência em contextos maiores.

Foi utilizada a divisão canônica proposta pelo *wikitext-2-raw-v1*, composta por partições de treino e validação sem remoção de pontuação ou normalização adicional de texto. A opção pela versão *raw* (em oposição à versão pré-processada) preserva melhor as características originais do corpus, incluindo quebras de linha e marcações mínimas, o que é mais alinhado ao uso típico de modelos autoregressivos de linguagem geral. Além disso, a utilização do *dataset* completo, em vez de uma amostragem reduzida, garante maior robustez estatística aos resultados de perplexidade e diminui a sensibilidade a flutuações devidas à escolha específica de exemplos.

O *pipeline* de preparação consistiu nas seguintes etapas:

1. **Tokenização integral de cada partição:** todos os textos das partições de treino e validação foram tokenizados com o *tokenizer* do GPT-2 (Seção 3.3), produzindo sequências de *input_ids* em formato numérico. Nessa etapa, não foram aplicados filtros de tamanho ou truncamentos individuais; os documentos foram convertidos integralmente em IDs de tokens.
2. **Concatenação dos *ids* e resegmentação em blocos fixos:** os *input_ids* de cada partição foram concatenados em um único fluxo contíguo de tokens, simulando um *stream* de texto contínuo. Em seguida, esse fluxo foi reparticionado em blocos de comprimento fixo de 1024 tokens (*block size*), descartando-se apenas os tokens remanescentes no final que não completassem um bloco. Essa estratégia de blocagem contígua é comum em modelagem de linguagem e evita desperdício de contexto em exemplos muito curtos, ao mesmo tempo em que expõe o modelo a transições naturais entre sentenças e parágrafos.
3. **Configuração de rótulos para modelagem autoregressiva:** para cada bloco de 1024 tokens, os *input_ids* foram copiados diretamente para o campo *labels*,

caracterizando o regime padrão de modelagem de linguagem autoregressiva, em que o modelo aprende a prever o próximo token dado o prefixo observado. Não foi utilizado *masking* especial nem deslocamentos adicionais além do comportamento já implementado pelo modelo de linguagem causal (o alinhamento entre *input_ids* e *labels* é tratado internamente pela cabeça de linguagem causal do GPT-2).

Após o processamento descrito, o *dataset* resultou em 2 318 amostras de treino e 240 amostras de validação, cada uma correspondente a um bloco de 1024 tokens. Esses valores refletem diretamente o tamanho da partição original do *wikitext-2-raw-v1* e o *block size* adotado, e são utilizados de forma consistente em todas as execuções experimentais, tanto nos *baselines* quanto nas variantes *PEFT*.

3.5 Métricas

A avaliação deste estudo considera duas dimensões principais: (i) qualidade preditiva, medida por perplexidade, e (ii) eficiência de inferência, medida por *throughput* em *tokens por segundo* e uso de memória de vídeo.

3.5.1 Qualidade preditiva: perplexidade

A qualidade dos modelos foi medida por perplexidade (*PPL*) em validação, métrica amplamente utilizada em modelagem de linguagem para quantificar o quão bem o modelo atribui probabilidade às sequências observadas. Seja $\mathcal{L}$ a perda média de entropia cruzada (*cross-entropy*) sobre o conjunto de validação, obtida a partir da cabeça de linguagem causal do GPT-2. A perplexidade é definida como

$$\text{PPL} = \exp(\mathcal{L}),$$

de modo que valores menores indicam melhor ajuste do modelo à distribuição de dados. Intuitivamente, a perplexidade pode ser interpretada como o número efetivo de alternativas equiprováveis que o modelo considera ao prever o próximo token: uma perplexidade de 30, por exemplo, sugere que o modelo, em média, se comporta como se estivesse escolhendo entre cerca de 30 opções plausíveis a cada passo.

Os detalhes operacionais do cálculo da perda, incluindo o uso de precisão *float32* na etapa de avaliação e a agregação das métricas em ambiente acelerado, seguem o protocolo unificado descrito na Seção 3.7.

3.5.2 Eficiência de inferência: *throughput* e uso de memória

A eficiência de inferência foi avaliada sob dois aspectos complementares:

1. **Tokens por segundo:** a taxa de *throughput* foi definida como o número total de tokens gerados em um experimento de inferência dividido pelo tempo de parede (*wall-clock time*) transcorrido. Essa medida captura o impacto combinado de simplificações arquiteturais (como a remoção de *LayerNorm*), do comprimento de sequência e do tamanho de *batch* sobre a velocidade efetiva de geração. O protocolo operacional de medição, incluindo *warmup*, sincronização explícita da GPU e configuração determinística de geração, é detalhado na Seção 3.8.
2. **Memória de vídeo máxima:** o uso de memória foi monitorado por meio da métrica de memória máxima alocada na GPU durante a inferência, obtida a partir das rotinas de monitoramento da biblioteca de *deep learning* (por exemplo, `torch.cuda.max_memory_allocated()`). Os valores brutos, em bytes, foram convertidos para gigabytes (GB) para fins de apresentação. Essa métrica indica o auge de consumo de VRAM por configuração de teste (modelo, comprimento de sequência, tamanho de *batch*), sendo relevante tanto para cenários de implantação em GPUs com capacidade limitada quanto para dimensionamento de recursos em ambientes de produção.

As duas métricas de eficiência foram sempre calculadas sobre o mesmo *pipeline* de geração, com *seed* fixa, configuração determinística de *sampling* (sem *temperature* ou *top-k/top-p*) e *tokenizer* comum às variantes. Ao reportar conjuntamente perplexidade, *tokens/s* e VRAM máxima, o estudo busca caracterizar o trade-off qualidade–desempenho–uso de memória para cada combinação de modelo (baseline vs. LN-Free) e técnica de adaptação *PEFT*, sob condições de inferência próximas a cenários práticos de uso.

3.6 Ambiente e controle experimental

Os experimentos foram executados em ambiente de GPU única, utilizando uma NVIDIA A100 com 40 GB de VRAM, em sessão baseada em *CUDA* 11 e *PyTorch* 2.x. A A100 oferece suporte nativo a operações em *bfloat16* (*bf16*) e a otimizações de atenção como *FlashAttention*, o que a torna adequada para experimentos de inferência e *fine-tuning* com modelos do porte do GPT-2 *small*. A orquestração de dispositivos, controle de precisão e agregação de métricas foi realizada por meio da biblioteca **accelerate**, da Hugging Face, que padroniza a preparação de modelos e *dataloaders* para execução em GPU com mínima modificação do código-base.

Com o objetivo de mitigar a variabilidade de *runtime* e favorecer a reprodutibilidade, foram adotadas as seguintes medidas de controle experimental:

- **Sementes pseudoaleatórias:** fixaram-se sementes para todas as bibliotecas relevantes (*Python*, *NumPy*, *PyTorch* e **accelerate**), utilizando a semente 1337 para os

baselines (A1, A2) e 42 para os treinos PEFT (A3–A6). Essa separação facilita a rastreabilidade dos experimentos e reduz a variância entre execuções distintas de um mesmo cenário, ainda que algum não determinismo permaneça devido a otimizações de baixo nível na GPU.

- **Precisão mista com *bfloat16*:** a precisão mista foi ativada em modo *bfloat16* (*bf16*) para acelerar o treino e reduzir o consumo de memória de vídeo nas execuções com GPU A100, explorando o suporte nativo do *hardware*. No *accelerate*, isso foi configurado por meio do parâmetro `mixed_precision='bf16'`. O uso de *bf16* reduz o custo de *forward* e *backward* mantendo uma faixa dinâmica ampla, o que é particularmente relevante em modelos sem *LayerNorm*. Uma exceção foi o método *BitFit* (A4), executado integralmente em *float32*, tanto por seu custo de treino já reduzido quanto para evitar possíveis efeitos numéricos sutis decorrentes de operar apenas sobre vieses em precisão mista.
- **Rotina padronizada de medição de inferência:** para medir eficiência de inferência (*tokens por segundo* e VRAM máxima), adotou-se uma rotina padronizada em todas as variantes. Cada bateria de testes incluiu (i) um período de aquecimento (*warmup*) para estabilizar a compilação de *kernels* e o agendamento da GPU; (ii) sincronização explícita do dispositivo (`torch.cuda.synchronize()`) imediatamente antes de iniciar a cronometragem e logo após o término da geração; e (iii) limpeza do *cache* de memória (`torch.cuda.empty_cache()`) ao final, antes da próxima configuração. Essa padronização reduz flutuações causadas por efeitos de *lazy initialization* e reutilização de *kernels* entre execuções.
- **Desativação de mapeamento automático de dispositivos:** desativou-se o mapeamento automático de dispositivos por camada fornecido por alguns utilitários da Hugging Face, assegurando que todo o modelo residisse explicitamente em uma única GPU A100. Dessa forma, evitam-se custos de comunicação adicionais e possíveis variações de *runtime* associadas a esquemas automáticos de *device mapping* (como particionamento entre GPU e CPU), garantindo que as comparações de eficiência reflitam apenas diferenças entre arquiteturas e métodos de adaptação, e não mudanças de *placement*.
- **Configuração consistente de *tokenizer* e *padding*:** assegurou-se que o `pad_token_id` fosse consistente com o `eos_token_id` em todos os modelos e que o *padding* fosse realizado à esquerda (`padding_side='left'`), conforme descrito na Seção 3.3. Essa configuração é essencial para que as rotinas de geração em *batch*, com *cache* de atenção ativado, operem de maneira idêntica entre as variantes baseline e LN-Free, evitando vieses na medição de *throughput* decorrentes de diferenças de alinhamento ou tratamento de *padding*.

Em conjunto, essas decisões visam criar um ambiente experimental controlado, no qual diferenças observadas em perplexidade, *tokens por segundo* e uso de memória possam ser atribuídas predominantemente às modificações arquiteturais (remoção de *LayerNorm*) e aos métodos de adaptação *PEFT*, e não a fatores externos como variações de inicialização, *overheads* de *hardware* ou inconsistências na preparação dos dados.

3.7 Protocolo de avaliação de perplexidade

A avaliação de perplexidade foi conduzida de forma padronizada para todas as variantes de modelo, utilizando o *framework* `accelerate` para orquestração em GPU. O procedimento visa garantir comparabilidade entre experimentos e estabilidade numérica, em especial na variante LN-Free. Para cada modelo, seguiram-se os passos:

1. **Alternância explícita para modo de avaliação:** o modelo era colocado em modo de avaliação por meio de `model.eval()`, desativando camadas sensíveis ao estado, como *dropout*, e assegurando um comportamento determinístico durante o *forward*. Toda a rotina de avaliação era envolvida em um bloco `torch.no_grad()`, evitando o armazenamento de gradientes e reduzindo o consumo de memória.
2. **Preparação com o *accelerator*:** o modelo e o *dataloader* de validação eram preparados pelo objeto `Accelerator` por meio de `accelerator.prepare(model, val_loader)`. Essa etapa cuida de mover o modelo e os lotes de dados para o dispositivo correto (GPU A100), aplicar a configuração de precisão mista (*bf16* quando habilitada) e, se necessário, lidar com cenários distribuídos. A partir desse ponto, o laço de avaliação passa a operar sobre instâncias já prontas para execução em GPU.
3. **Cálculo da perda em precisão *float32*:** para cada lote de validação, calculou-se a perda de entropia cruzada ($\mathcal{L}$) utilizando a cabeça de linguagem causal do GPT-2. Embora o modelo pudesse estar em *bfloat16* ou *float16* para o restante do *forward*, o cálculo da *loss* foi forçado a *float32* por meio de um contexto de autocasting apropriado, a fim de garantir estabilidade numérica, principalmente na variante LN-Free. Especificamente, o valor de $\mathcal{L}$ era computado em cada lote, convertido para `torch.float32` e, em seguida, agregado via `accelerator.gather_for_metrics` para reunir as contribuições de todos os processos (mesmo no caso de execução em GPU única). Ao final da passagem sobre o conjunto de validação, obtinha-se a perda média global:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N \mathcal{L}_i,$$

em que N é o número de lotes de validação e $\mathcal{L}_i$ é a perda do lote i .

4. **Transformação da perda média em perplexidade:** de posse da perda média $\mathcal{L}$, a perplexidade foi calculada pela relação padrão

$$\text{PPL} = \exp(\mathcal{L}),$$

conforme descrito na Seção 3.5. Esse valor escalar foi então registrado como métrica de qualidade para o respectivo modelo e configuração de treino, sendo utilizado nas comparações entre o baseline (A1, A2) e as variantes *PEFT* (A3–A6).

No caso de modelos adaptados por *PEFT* do tipo *LoRA* e *AdaLoRA*, realizou-se um *merge* das deltas nos pesos do modelo-base antes da fase de avaliação e das medições de inferência. Em termos práticos, os incrementos de baixo ranque aprendidos ($BA^\top$) eram fundidos nas matrizes originais de projeção, resultando em um único conjunto de pesos efetivos para a etapa de geração. Essa escolha garante que a perplexidade e o *throughput* reflitam o custo real do modelo implantado (sem operadores adicionais em tempo de execução) e torna as comparações mais justas com as variantes baseline e com métodos como *BitFit* e *IA*³, que já modificam apenas o caminho padrão de inferência sem exigir estruturas auxiliares em tempo de execução.

3.8 Benchmark de inferência

O *benchmark* de inferência teve como objetivo comparar diretamente o custo de geração entre o modelo baseline (GPT-2 *small*) e a variante LN-Free, em condições controladas e próximas ao uso prático. Em todos os cenários, foi utilizada geração determinística, com máscaras causais padrão de modelos de linguagem autoregressivos e *cache* de atenção (*key-value caching*) ativado, de modo que a cada novo token gerado apenas as novas chaves e valores fossem computados, reaproveitando os estados armazenados dos tokens anteriores. Esse procedimento reflete a forma usual de implantação de LLMs em produção, na qual o uso de *KV cache* reduz o custo por passo em sequências longas.

Para cada medição de *throughput*, seguiu-se o seguinte protocolo:

1. **Aquecimento (*warmup*):** executou-se inicialmente uma chamada curta à função de geração, com poucas etapas de *decode*, descartando os tempos medidos. Essa fase de aquecimento permite que a GPU carregue os *kernels* necessários, que a pilha de execução estabilize eventuais otimizações internas e que efeitos de *lazy initialization* sejam amortizados antes da cronometragem efetiva.
2. **Medição do tempo de geração:** em seguida, mediu-se o tempo de parede (*wall-clock time*) para gerar um número fixo de novos tokens (128) a partir de *inputs* já preenchidos com o `eos_token`. Para isso, foram construídos tensores de entrada

com comprimento de sequência pré-definido, compostos inteiramente pelo token de fim de sequência, simulando um contexto saturado em que o custo de atenção se torna mais relevante. Antes do início da cronometragem, a GPU era explicitamente sincronizada com `torch.cuda.synchronize()`, e o mesmo comando era chamado logo após o término da geração, garantindo que o tempo medido correspondesse apenas às operações de inferência.

3. **Cálculo de *tokens por segundo*:** com o tempo total decorrido e o número de tokens gerados, computou-se a métrica de *tokens por segundo* (*throughput*) pela razão entre o total de tokens gerados e o tempo medido. No caso de *batch size* maior que 1, o número total de tokens considera a soma sobre todas as sequências do *batch*. Essa métrica quantifica diretamente a velocidade efetiva de geração sob a combinação de arquitetura (com ou sem *LayerNorm*) e configurações de *batch* e comprimento de sequência.

Na tabela principal de *runs* (Tabela 2), adotou-se *batch size* igual a 1 e comprimento de sequência de 512 tokens na entrada, gerando sempre 128 tokens adicionais. Esse cenário representa um uso típico de interação pontual (*single request*) com contexto relativamente longo, sendo adequado para comparar a eficiência de inferência entre o GPT-2 *small* oficial e a variante LN-Free sem interferência de efeitos de *batching*.

Para caracterizar mais amplamente o comportamento de *throughput*, realizou-se ainda um *sweep* de inferência (Tabela 4) variando sistematicamente o comprimento de sequência e o tamanho de *batch*. Foram avaliadas combinações

$$(seq, batch) \in \{256, 512, 896\} \times \{1, 8\},$$

mantendo o número de tokens gerados por sequência fixo (128). Os comprimentos de 256, 512 e 896 tokens foram escolhidos para cobrir desde contextos moderados até próximos ao limite do *block size* de 1024, enquanto os tamanhos de *batch* 1 e 8 permitem comparar cenários de requisições isoladas e requisições concorrentes agrupadas. Esse *sweep* permite observar como a vantagem relativa do modelo LN-Free se comporta à medida que aumentam o custo quadrático da atenção e a ocupação da GPU, fornecendo uma visão mais completa do trade-off entre arquitetura e eficiência em regimes de inferência distintos.

3.9 Métodos de adaptação avaliados

Todos os métodos *PEFT* foram aplicados sobre ambos os modelos *baseline* (GPT-2 oficial e LN-Free) sob condições de treino idênticas, incluindo *dataset*, número de passos (300), *batch size* efetivo, *scheduler* e taxa de aprendizado. Em todos os casos, os pesos originais do modelo permaneceram congelados, e apenas os parâmetros adicionais introduzidos por cada técnica foram atualizados, conforme descrito a seguir.

3.9.1 *LoRA*

Na adaptação via *LoRA*, empregou-se uma configuração base com posto de baixo ranque $r = 8$, fator de escalonamento $\alpha = 16$ e *dropout* de 0,05 aplicado sobre as atualizações de baixo ranque. Essa configuração implica uma razão de $\alpha/r = 2$, que controla a amplitude efetiva do desvio aplicado sobre as projeções originais.

Os alvos da *LoRA* foram os módulos lineares `c_attn`, `c_proj` e `c_fc`, correspondentes, respectivamente, às projeções de atenção (QKV), à projeção de saída da atenção e à primeira projeção da rede *feed-forward* em cada bloco Transformer. Em cada um desses módulos, a matriz de pesos original W foi mantida congelada e expandida com um incremento de baixo ranque $BA^\top$, em que A e B são matrizes treináveis de dimensão reduzida. A orientação de *fan-in fan-out* foi configurada de forma compatível com as convenções de projeção do GPT-2, garantindo que a decomposição de baixo ranque incida sobre a dimensão correta dos tensores internos.

Essa escolha de alvos busca equilibrar capacidade de adaptação e custo paramétrico: ao atuar simultaneamente sobre projeções de atenção e sobre a parte densa da MLP, a *LoRA* dispõe de graus de liberdade suficientes para compensar alterações estruturais (como a remoção de *LayerNorm*) sem introduzir um número excessivo de parâmetros adicionais. Após o término do treino, as deltas aprendidas ($BA^\top$) foram fundidas nas matrizes originais (*merge*) antes das medições de perplexidade e *throughput*, de forma que o caminho de inferência final não incluísse camadas auxiliares específicas de *LoRA*.

3.9.2 *BitFit*

No método *BitFit*, apenas os parâmetros de viés (*bias*) permaneceram treináveis, enquanto todos os demais pesos do modelo foram congelados. Em termos práticos, isso inclui vieses em camadas lineares das projeções de atenção, das redes *feed-forward* e, quando presentes, vieses em camadas de saída. Nenhuma nova estrutura foi adicionada ao modelo; trata-se de um ajuste fino minimalista sobre parâmetros já existentes.

O treino desse método foi executado integralmente em precisão *float32*, sem uso de precisão mista, tanto por seu custo computacional já reduzido (devido ao baixo número de parâmetros atualizados) quanto para evitar impactos de arredondamento em um regime em que o gradiente é propagado apenas por um subconjunto muito pequeno de parâmetros. O *BitFit* serve, assim, como linha de base de custo mínimo de adaptação, permitindo avaliar quanto da degradação decorrente da remoção de *LayerNorm* pode ser recuperada apenas ajustando vieses.

3.9.3 AdaLoRA

A *AdaLoRA* foi configurada com parâmetros iniciais equivalentes aos da *LoRA* ($r = 8$, $\alpha = 16$), utilizando o mesmo conjunto de módulos-alvo (`c_attn`, `c_proj`, `c_fc`). A diferença central em relação à *LoRA* está no mecanismo adaptativo de alocação de baixo ranque: em vez de manter o ranque r fixo em todas as camadas ao longo de todo o treino, a *AdaLoRA* estima a importância de cada matriz adaptada e redistribui dinamicamente o orçamento de ranque ao longo das etapas de treinamento.

Para isso, o método recebe o orçamento total de passos de treino (`total_step`), informado como 300, em sincronia com o número máximo de passos efetivamente utilizados neste trabalho. Com base em estatísticas coletadas durante o treino (por exemplo, magnitudes de gradientes ou valores singulares associados às atualizações), a *AdaLoRA* conserva componentes mais importantes e poda gradualmente componentes menos relevantes, concentrando capacidade de baixo ranque em submódulos mais sensíveis. O objetivo é aproveitar melhor o mesmo orçamento paramétrico inicial da *LoRA*, potencialmente obtendo recuperação de qualidade superior em cenários em que a distribuição de importância entre camadas é desigual. Assim como na *LoRA*, as deltas finais são fundidas nos pesos do modelo antes da avaliação de inferência.

3.9.4 IA³

Na adaptação via IA³, não são introduzidas matrizes de baixo ranque adicionais, mas sim tensores de escalonamento multiplicativo que modulam subespaços das ativações internas. Neste trabalho, esses tensores foram aplicados a três grupos de submódulos: projeções de atenção (`c_attn`), projeções de saída da atenção (`c_proj`) e camadas *feed-forward* (`c_fc`), com configuração de *fan-in fan-out* compatível com a implementação do GPT-2.

Em termos conceituais, a IA³ aprende vetores (ou pequenas matrizes) de pesos que reescalonom multiplicativamente determinados eixos das ativações, por exemplo, multiplicando componentes de chaves, valores ou saídas intermediárias da MLP por fatores aprendidos, específicos da tarefa. Como os pesos densos originais permanecem congelados, o número de parâmetros adicionais é extremamente reduzido quando comparado à *LoRA* ou à *AdaLoRA*. Além disso, o custo computacional extra na inferência resume-se a algumas operações de multiplicação elemento a elemento, sem alteração da forma das matrizes principais ou da complexidade assintótica da atenção e das camadas densas.

Essa característica torna a IA³ particularmente atraente em cenários em que se deseja minimizar não apenas o custo de treino, mas também o impacto no caminho de inferência, mantendo o modelo resultante muito próximo, em termos de estrutura e custo, ao modelo base. Em combinação com a remoção de *LayerNorm*, a IA³ fornece um

mecanismo de ajuste fino de baixo custo que pode compensar parcialmente a perda de estabilidade introduzida pela simplificação arquitetural sem reintroduzir operações de normalização.

3.10 Hiperparâmetros de treino

Para garantir uma comparação direta e controlada entre os diferentes métodos de adaptação *PEFT* (LoRA, BitFit, AdaLoRA e IA³), todos foram treinados sob um conjunto unificado de hiperparâmetros, utilizando o *framework accelerate* para orquestração em GPU. O objetivo dessa padronização é assegurar que as diferenças observadas em perplexidade e eficiência de inferência sejam atribuíveis principalmente à técnica de adaptação empregada, e não a variações de configuração de treino.

Os hiperparâmetros adotados foram:

- **Número de passos (steps):** todos os métodos foram treinados por 300 passos de atualização. Esse horizonte relativamente curto é suficiente para observar a capacidade de recuperação de qualidade dos métodos *PEFT* em cenário de *low budget*, mantendo ao mesmo tempo o custo de experimento compatível com os recursos disponíveis. Além disso, a escolha de um número fixo de passos para todos os métodos evita que qualquer técnica seja favorecida por maior exposição ao conjunto de treino.
- **Taxa de aprendizado (learning rate):** utilizou-se uma taxa de aprendizado global de 2×10^{-4} (0,0002) para todos os métodos. Esse valor situa-se em uma faixa típica para adaptações leves em modelos pré-treinados de porte médio, oferecendo um bom compromisso entre velocidade de convergência e estabilidade. A uniformização da taxa de aprendizado facilita a comparação entre LoRA, BitFit, AdaLoRA e IA³, reduzindo o espaço de busca de hiperparâmetros.
- **Aquecimento (warmup):** os primeiros 25 passos foram dedicados ao aquecimento linear da taxa de aprendizado, partindo de valores próximos de zero até atingir a taxa máxima de 2×10^{-4} . Essa fase de aquecimento mitiga instabilidades iniciais, especialmente em cenários em que apenas um subconjunto reduzido de parâmetros é treinado, e é recomendada em treinamentos sobre modelos grandes pré-treinados.
- **Agendador (scheduler):** após o período de *warmup*, a taxa de aprendizado passou a seguir um agendador do tipo cosseno (*cosine decay*), decaindo suavemente até o final dos 300 passos. Esse esquema de *scheduler* permite que o treino explore uma taxa relativamente alta na fase intermediária e, gradualmente, reduza a intensidade das atualizações perto do final, favorecendo a estabilização das métricas de validação.

- **Otimizador (optimizer):** em todas as execuções foi utilizado o otimizador *AdamW*, que combina o método Adam com regularização do tipo *weight decay* desacoplado. O *AdamW* é amplamente empregado em *fine-tuning* de modelos de linguagem, por lidar bem com gradientes esparsos e por oferecer convergência estável mesmo quando apenas uma fração dos parâmetros é atualizada.
- **Batch size por dispositivo:** o *batch size* por dispositivo (GPU) foi fixado em 8 exemplos por passo. Esse valor explora de forma razoável a capacidade de memória da A100 (40 GB) no contexto dos modelos considerados, sem incorrer em *out-of-memory*, e mantém as estatísticas de gradiente suficientemente estáveis, mesmo para um número moderado de passos totais.
- **Acumulação de gradiente:** empregou-se acumulação de gradiente por 4 passos consecutivos (*gradient accumulation steps* = 4). Em termos efetivos, a cada 4 *forwards* com *batch* de 8 exemplos, os gradientes eram acumulados antes de se realizar uma única atualização dos parâmetros. Isso equivale a um *batch size* efetivo maior, sem exigir que todos os exemplos caibam simultaneamente na memória da GPU.
- **Batch size efetivo:** considerando o *batch size* por dispositivo (8) e a acumulação de gradiente em 4 passos, o *batch size* efetivo utilizado nos cálculos de atualização é 32 exemplos por passo de otimização (8×4). Esse valor é um compromisso entre eficiência estatística (gradientes menos ruidosos) e custo computacional, e é mantido constante entre todos os métodos avaliados.

Essa padronização de hiperparâmetros, associada ao uso de um único ambiente de hardware e ao mesmo *pipeline* de dados, permite isolar o efeito de cada método *PEFT* sobre o desempenho dos modelos baseline e LN-Free, mantendo constante o orçamento de treino (número de passos, taxa de aprendizado e *batch size* efetivo).

3.11 Tratamento da variante *LN-Free*

No GPT-2 *small* original, cada bloco Transformer contém duas camadas de normalização (`ln_1` e `ln_2`), aplicadas em torno dos subblocos de atenção e da rede *feed-forward*, além de uma normalização final `ln_f` sobre a saída do último bloco. Na variante *LN-Free*, essas operações são removidas da trajetória de inferência, de modo que o caminho *forward* deixe de incluir normalizações explícitas e passe a depender apenas de projeções lineares, funções de ativação e conexões residuais.

A remoção de *LayerNorm* foi implementada substituindo-se as instâncias dos módulos `ln_1`, `ln_2` e `ln_f` por identidades (`nn.Identity`) no objeto de modelo utilizado na etapa de inferência. Em termos práticos, isso equivale a “curto-circuitar” as chamadas

às camadas de normalização no grafo de execução, eliminando operações ponto a ponto e movimentações de memória associadas à normalização, sem alterar a forma dos tensores intermediários. Essa simplificação é compatível com a forma como os *checkpoints* LN-Free públicos são gerados a partir do GPT-2, via remoção de *LayerNorm* seguida de um curto *fine-tuning* corretivo.

Para efeito de comparabilidade na Tabela 2, a variante *LN-Free* foi avaliada em dois cenários distintos:

1. **raw (A2a)**: medição de perplexidade e *throughput* diretamente no *checkpoint* LN-Free original, tal como disponibilizado no repositório público (<schaeff/gpt2-small_LNFree300>), sem qualquer modificação estrutural adicional no código do modelo. Esse cenário reflete o comportamento “de prateleira” do modelo LN-Free, inclusive eventuais decisões de implementação do autor do *checkpoint*.
2. **identity (A2b)**: remoção explícita das normalizações no objeto de modelo carregado, substituindo os módulos `ln_1`, `ln_2` e `ln_f` por `nn.Identity` antes da medição de *throughput*. Nesse caso, o modelo preserva os pesos do *checkpoint* LN-Free (já ajustados via *fine-tuning*), mas passa a operar com um grafo de inferência estritamente livre de normalizações, permitindo estimar o ganho de velocidade diretamente atribuível à ausência de *LayerNorm* no caminho *forward*. As demais configurações (tokenização, tamanho de sequência, *batch*, precisão numérica) são mantidas idênticas às do cenário A2a.

Essa distinção entre A2a e A2b permite separar dois aspectos: (i) o efeito global de se adotar um *checkpoint* LN-Free treinado com remoção de normalizações (A2a), incluindo eventuais diferenças de implementação em relação ao GPT-2 oficial; e (ii) o impacto específico de eliminar as chamadas às camadas de normalização no grafo de execução, quando todas as demais condições são controladas (A2b).

Sobre essa base, os métodos *PEFT* (A3–A6) foram então aplicados diretamente ao *checkpoint* `schaeff/gpt2-small_LNFree300`. Em outras palavras, cada técnica de adaptação leve partiu de um modelo já ajustado à ausência de *LayerNorm*, buscando recuperar ou melhorar a qualidade preditiva sem reintroduzir normalizações no caminho de inferência. Isso assegura que os ganhos de eficiência observados na variante LN-Free sejam preservados, enquanto as técnicas *PEFT* atuam apenas como mecanismos de correção paramétrica sobre a arquitetura simplificada.

3.12 Sweep de inferência

Para caracterizar de forma mais abrangente o impacto da simplificação arquitetural na prática, foi conduzido um *sweep* sistemático de inferência comparando o *baseline* oficial (A1) e a versão *LN-Free* com identidades substitutas (A2b). Ao contrário dos testes pontuais da Tabela 2, que utilizam apenas uma configuração de sequência e *batch*, o *sweep* explora diferentes regimes de carga sobre o modelo, permitindo observar como os ganhos de *throughput* variam com o comprimento de sequência e o tamanho do *batch*.

Foram avaliadas combinações de $(seq, batch)$ descritas na Seção 3.8, a saber:

$$(seq, batch) \in \{256, 512, 896\} \times \{1, 8\}.$$

O comprimento de sequência *seq* corresponde ao número de tokens de contexto já presentes na entrada (antes da geração), enquanto o *batch size* indica quantas sequências são processadas em paralelo pela GPU. Em todas as configurações, o número de tokens gerados por sequência foi mantido fixo (128), com *cache* de atenção ativado e geração determinística, seguindo o protocolo descrito na Seção 3.8.

Em cada ponto do *sweep*, foram executadas múltiplas medições de tempo de geração para o GPT-2 *small* oficial (A1) e para a variante LN-Free com normalizações substituídas por identidades (A2b). A partir desses tempos, computou-se o *throughput* em *tokens por segundo*, tomando-se a razão entre o total de tokens gerados (número de sequências no *batch* multiplicado pelos 128 tokens gerados por sequência) e o tempo de parede medido, após sincronização da GPU. As leituras foram agrupadas na Tabela 4, que consolida, para cada par $(seq, batch)$, o *throughput* observado para A1 e A2b.

Essa organização permite discutir o perfil de ganhos da variante LN-Free sob diferentes regimes: por exemplo, comparar o benefício relativo em contextos curtos ($seq = 256$) versus contextos longos ($seq = 896$), assim como a influência de processar requisições isoladas ($batch = 1$) em contraste com requisições agrupadas ($batch = 8$). Em particular, o *sweep* evidencia se o ganho de velocidade decorrente da remoção de *LayerNorm* cresce com o comprimento de sequência (como esperado, dado o custo acumulado de operações ponto a ponto) e se tal ganho se mantém ou se dilui quando a GPU é mais bem aproveitada por meio de *batching* maior. Esses resultados servem de base para as análises comparativas apresentadas no capítulo de resultados.

3.13 Armazenamento e organização de resultados

Todos os resultados numéricos produzidos pelos experimentos foram persistidos em disco em formatos tabulares, de modo a facilitar tanto a geração de tabelas no texto quanto a auditoria e reuso posterior. Foram geradas, em particular, três tabelas principais em formato *CSV*, acompanhadas de versões equivalentes em *JSON*, com o seguinte conteúdo:

- **summary_baselines.csv**: contém o resumo dos *baselines* A1, A2a e A2b, incluindo métricas de qualidade (perplexidade em validação), *throughput* de inferência (tokens por segundo) e uso máximo de memória de vídeo (VRAM), bem como metadados relevantes (identificador do modelo, configuração de sequência e *batch* empregadas na medição).
- **summary_peft_training.csv**: registra, para cada combinação de modelo base (GPT-2 oficial ou LN-Free) e método *PEFT* (A3–A6), as métricas obtidas ao final do treino: perda e perplexidade em validação, número de parâmetros treináveis, fração de parâmetros em relação ao total do modelo e, quando aplicável, indicadores de estabilidade do treino (por exemplo, divergência em AdaLoRA). Essa tabela sintetiza o efeito de cada técnica de adaptação sob o orçamento de 300 passos.
- **bench_inference_sweep.csv**: armazena os resultados do *sweep* de inferência descrito na Seção 3.12, incluindo, para cada par (*seq*, *batch*) e para cada modelo (A1, A2b), as métricas de *throughput* (*tokens/s*) e VRAM máxima. Cada linha identifica explicitamente o modelo, o comprimento de sequência, o *batch size* e as condições de geração utilizadas.

Os arquivos no formato JSON preservam, além das métricas agregadas, informações brutas das execuções (por exemplo, tempos individuais de rodadas de *benchmark*, *seeds* utilizadas e configurações internas do *accelerate*), permitindo auditoria e reprodutibilidade mais fina dos experimentos. Esses artefatos foram organizados em um diretório único de saída, acessado pelo código de geração das tabelas do capítulo de resultados, o que reduz o acoplamento entre código e texto e facilita a atualização de quadros caso novos experimentos sejam incorporados no futuro. Todos esses arquivos (CSV e JSON), bem como o notebook completo e o script Python simplificado, encontram-se versionados no repositório público associado a este trabalho (CANDIDO, 2025).

3.14 Instruções de reprodutibilidade

Esta seção resume, em nível conceitual, os passos necessários para reproduzir localmente os experimentos descritos neste trabalho, sem se prender a um ambiente específico (como Google Colab). O objetivo é indicar os requisitos mínimos de hardware/software e o fluxo geral de execução, remetendo ao repositório público para detalhes operacionais.

Requisitos de ambiente

Os experimentos foram executados em ambiente com GPU NVIDIA A100 (40 GB VRAM), suporte a CUDA 11+ e PyTorch recente, mas podem ser reproduzidos em qualquer sistema que disponha de GPU compatível com CUDA e memória suficiente

para o GPT-2 small nos modos de precisão utilizados. A Tabela 1 apresenta as versões recomendadas das principais bibliotecas.

Tabela 1 – Versões de bibliotecas recomendadas para reprodutibilidade.

Biblioteca	Versão utilizada
Python	3.10.12
torch	2.3.0 ou superior (com suporte a CUDA)
transformers	4.42.0
datasets	2.20.0
peft	0.12.0
accelerate	0.30.0+
safetensors	0.4.2
evaluate	0.4.3

Fonte: Elaboração do autor (2025).

Na prática, basta criar um ambiente isolado (por exemplo, via `virtualenv` ou `conda`) com Python 3.10+ e instalar as dependências listadas na Tabela 1, garantindo que a versão de `torch` seja compatível com a GPU disponível.

Fluxo geral de execução

O fluxo de reprodutibilidade segue, em alto nível, as seguintes etapas:

- Preparar o ambiente:** criar um ambiente virtual Python 3.10+ e instalar `torch`, `transformers`, `datasets`, `peft`, `accelerate`, `safetensors` e `evaluate`—em versões compatíveis com a Tabela 1. Configurar variáveis como `TOKENIZERS_PARALLELISM=false` e `WANDB_DISABLED=true`.
- Obter o código e os artefatos:** clonar o repositório público associado a este trabalho (CANDIDO, 2025). O repositório contém: (i) o notebook principal dos experimentos, (ii) o script Python simplificado (Seção 3.15), (iii) todos os arquivos de resultados (`summary_*.csv`, `bench_inference_sweep.csv` e respectivos `.json`).
- Executar o pipeline:** rodar o notebook ou o script principal em GPU compatível, seguindo o mesmo fluxo descrito no Capítulo 3: carregamento do dataset `wikitext-2-raw-v1`, carregamento dos modelos `baseline` e `LN-Free`, execução dos treinos PEFT (A3–A6), avaliação de perplexidade (Seção 3.7) e execução dos benchmarks de inferência (Seções 3.8 e 3.12). As tabelas do Capítulo 4 são geradas automaticamente a partir dos arquivos CSV.

Detalhes operacionais (como comandos específicos, divisão de células do notebook ou instruções específicas de Colab) ficam documentados apenas no repositório, pois não afetam o delineamento metodológico.

Os scripts, notebooks e artefatos deste trabalho são disponibilizados sob licença MIT no repositório público (CANDIDO, 2025).

3.15 Script *Python* resumido para reprodutibilidade

Esta seção apresenta um script *Python* autocontido, resumido a partir do *pipeline* principal (baseado em `accelerate`), com a finalidade de reprodutibilidade mínima. Ele ilustra o carregamento do modelo *baseline*, a avaliação de *perplexity* e um *benchmark* simples de geração, refletindo a metodologia unificada.

```
import os, math, time, random, json, torch, numpy as np
import torch.nn as nn
from datasets import load_dataset
from torch.utils.data import DataLoader
from accelerate import Accelerator
from accelerate.utils import set_seed as accelerate_set_seed
from transformers import (AutoTokenizer, AutoModelForCausalLM,
                          default_data_collator, AutoConfig)

# --- 1. Configuracao do Ambiente ---
os.environ.setdefault("TOKENIZERS_PARALLELISM", "false")
os.environ.setdefault("WANDB_DISABLED", "true")

# Configuracao unificada (exemplo)
CFG = {
    "block_size": 1024,
    "per_device_eval_batch_size": 8,
}
MODEL_ID = "gpt2"
SEED = 1337

# --- 2. Setup do Accelerator ---
# Define a precisao (bf16 para A100, fp16 para T4/V100)
has_bf16 = torch.cuda.is_available() and torch.cuda.is_bf16_supported()
precision = "bf16" if has_bf16 else "fp16"
accelerator = Accelerator(mixed_precision=precision)
DEVICE = accelerator.device

def set_seed(seed=42):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed);
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)
    accelerate_set_seed(seed)

set_seed(SEED)
```



```

# --- 3. Carregamento de Dados (Simplificado) ---
# (Aqui entraria a funcao 'load_unified_dataset' completa)
# tok, train_loader, val_loader = load_unified_dataset(CFG, "gpt2")
# Para este exemplo, supomos que 'val_loader' existe
# e 'tok' foi carregado com tok.padding_side = 'left'

# --- 4. Funcoes de Avaliacao (do Notebook) ---
def compute_loss_on_batch(model, batch):
    out = model(input_ids=batch["input_ids"],
                 attention_mask=batch.get("attention_mask"),
                 labels=batch["labels"])
    return out.loss

def eval_loop_accelerate(model, val_loader):
    model.eval()
    losses = []
    with torch.no_grad():
        for i, batch in enumerate(val_loader):
            # Forca calculo da loss em FP32 para estabilidade
            with torch.autocast(device_type="cuda", dtype=torch.float32):
                loss = compute_loss_on_batch(model, batch)
            gathered = accelerator.gather_for_metrics(loss.detach().reshape(1))
            losses.append(gathered.mean().cpu())

    mean_loss = torch.stack(losses).mean().item()
    ppl = math.exp(mean_loss) if (mean_loss < 20) else float("inf")
    model.train()
    return mean_loss, ppl

# --- 5. Carregamento do Modelo (Baseline A1) ---
def load_model_simple(model_id, device="cuda"):
    model = AutoModelForCausalLM.from_pretrained(
        model_id, dtype=torch.float32 # Carrega em FP32
    )
    if torch.cuda.is_available():
        # Converte para dtype rapido na GPU
        dtype = torch.bfloat16 if torch.cuda.is_bf16_supported() else torch.float16
        model = model.to(device=device, dtype=dtype)
    return model

print(f"Rodando Baseline A1: {MODEL_ID}")
model_a1 = load_model_simple(MODEL_ID, device=DEVICE)

# --- 6. Execucao da Avaliacao de PPL ---
# (O codigo abaixo depende da criacao real do val_loader)
# print("Preparando modelo e dataloader com accelerator...")
# model_a1_prep, val_loader_prep = accelerator.prepare(model_a1, val_loader)

```

```
# print("Avaliando PPL...")
# loss, ppl = eval_loop_accelerate(model_a1_prep, val_loader_prep)
# print(f"--- A1 Concluído --- Loss: {loss:.4f}, PPL: {ppl:.2f}")

# (Limpeza)
# del model_a1, model_a1_prep, val_loader_prep
# if torch.cuda.is_available(): torch.cuda.empty_cache()
```

3.16 Metodologia detalhada de avaliação

Esta seção complementa, de forma sintética, as descrições metodológicas anteriores, enfatizando a organização dos experimentos e os critérios gerais de comparação, sem repetir detalhes já apresentados nas seções deste capítulo.

Descrição geral dos experimentos

Todos os experimentos foram conduzidos em ambiente Google Colab com GPU NVIDIA A100 (40 GB de VRAM), Python 3.10 e bibliotecas *Transformers*, *PEFT* e *accelerate*, em linha com o ambiente descrito na Seção 3.6. As execuções foram organizadas por método de adaptação *PEFT* (LoRA, BitFit, AdaLoRA e IA³) e pelos *baselines*, produzindo arquivos de log, tabelas e métricas numéricas. Todos os artefatos foram exportados para o diretório `out_unified_experiment_a100/` e consolidados conforme descrito na Seção 3.13.

Controle de aleatoriedade

O controle de aleatoriedade seguiu o esquema de sementes descrito em 3.6, com valores distintos para *baselines* e treinos PEFT. Em conjunto com o uso de um único ambiente de hardware, isso reduz variações estatísticas entre execuções, ainda que alguma não determinismo permaneça devido a otimizações específicas da GPU.

Pré-processamento e tokenização

O pré-processamento e a tokenização do corpus *wikitext-2-raw-v1* seguiram exatamente o *pipeline* descrito nas Seções 3.3 e 3.4, incluindo o uso de blocos de 1024 tokens e da versão completa do *dataset*. Os *DataLoaders* foram construídos com a biblioteca *datasets* e integrados ao *accelerate* para uso em GPU.

Métricas de avaliação

As métricas principais — perplexidade, *throughput*, uso de VRAM e número de parâmetros treináveis — foram definidas e operacionalizadas nas Seções 3.5, 3.7, 3.8 e 3.12.

A perplexidade foi obtida a partir do *loop* manual de avaliação, com cálculo de *loss* em *float32*; o *throughput* e o uso de VRAM foram medidos com o protocolo de *benchmark* de inferência, sempre sob configuração determinística de geração.

Organização dos resultados e armazenamento

A organização dos resultados experimentais em arquivos `.csv` e `.json`, bem como a estrutura do diretório `out_unified_experiment_a100/`, segue a descrição detalhada da Seção 3.13. Essa organização foi projetada para permitir a reconstrução das tabelas do capítulo de resultados e auditoria independente das execuções.

Crítérios de comparação e análise

As comparações entre modelos e métodos de adaptação tomaram como referência o GPT-2 *small* oficial (A1), avaliando-se ganhos relativos de eficiência (variação percentual de *throughput* e uso de VRAM) e variações de qualidade (diferença e razão de perplexidade). A interpretação dos resultados considerou um critério de Pareto entre custo e desempenho: métodos que reduziram significativamente o custo de inferência sem aumento substancial de perplexidade foram classificados como mais eficientes. As análises quantitativas e qualitativas decorrentes desses critérios são apresentadas no capítulo de resultados e discussão.

Considerações adicionais

Os experimentos foram executados uma vez sob um ambiente unificado e controlado, sem aplicação de técnicas adicionais de quantização ou compressão, de forma a isolar o efeito das técnicas de adaptação e da remoção de normalização. As limitações decorrentes desse delineamento — como horizonte de treino reduzido, uso de um único conjunto de dados e foco em um único porte de modelo — são discutidas na Seção 3.17 e retomadas na conclusão do trabalho.

3.17 Limitações metodológicas

As limitações principais deste delineamento incluem: horizonte de treino reduzido (300 passos) nos métodos de adaptação, o que se mostrou insuficiente para a convergência do *AdaLoRA*; variação inerente ao *runtime* de ambientes compartilhados; uso de um único conjunto de dados para avaliação de perplexidade; e foco em um único porte de modelo. Tais limitações são consideradas na discussão e endereçadas nas propostas de trabalhos futuros.

4 Resultados

Este capítulo apresenta os resultados experimentais consolidados para o *baseline* oficial do GPT-2 *small* (A1) e para a variante sem *LayerNorm* (A2, LN-Free), bem como para os quatro métodos de adaptação parametricamente eficiente (A3-A6) aplicados sobre ambos os modelos.

Todos os experimentos de treinamento e avaliação de perplexidade (PPL) foram executados sob um conjunto unificado de hiperparâmetros (300 passos, LR 2e-4, *batch* efetivo de 32) e dados (dataset *Wikitext* completo, *block size* 1024), garantindo comparabilidade direta. Reportam-se PPL em validação, taxa de geração em *tokens* por segundo (toks/s) e uso máximo de memória de vídeo (VRAM).

4.1 Resumo dos experimentos

A seguir são apresentados, de forma consolidada, os resultados quantitativos obtidos nos experimentos executados ao longo do estudo. Os valores abrangem tanto os modelos base (A1 e A2) quanto os métodos de adaptação leve (A3–A6), permitindo uma análise comparativa direta de qualidade, eficiência e economia de parâmetros treináveis.

A Tabela 2 apresenta as métricas correspondentes aos modelos base antes da aplicação de qualquer método PEFT. Esses resultados estabelecem o ponto de partida para a interpretação dos ganhos ou perdas observados após o *fine-tuning*.

Tabela 2 – Resultados dos Baselines (A1 e A2) — avaliação de PPL e inferência (*single-batch*, 512 tokens) no dataset *Wikitext* completo.

Run	Modelo	Loss	PPL	toks/s	VRAM (GB)
A1	gpt2 (oficial)	3,43	30,88	83,24	0,30
A2a	gpt2-ln-free (raw)	3,60	36,54	84,25	0,55
A2b	gpt2-ln-free (identity)	3,60	36,54	105,28	0,55

Fonte: Elaboração do autor (2025).

A Tabela 3 sumariza os resultados completos do treinamento PEFT para os quatro métodos avaliados (LoRA, BitFit, AdaLoRA e IA³), aplicados tanto ao GPT-2 oficial quanto à variante LN-Free. São apresentados valores finais de loss, perplexity, número de parâmetros treináveis e a fração relativa do modelo ajustada em cada método.

Tabela 3 – Resultados do *fine-tuning* PEFT (A3–A6) após 300 passos de treino no dataset *Wiki-text* (tamanho de lote efetivo = 32, taxa de aprendizado = 2e-4).

Modelo	Método (PEFT)	Loss Final	PPL Final	Parâms. Treináveis	% Treinável
Modelo: gpt2 (Baseline PPL: 30.88)					
gpt2	LoRA (A3)	3,28	26,64	1 179 648,00	0,94
gpt2	BitFit (A4)	3,34	28,18	102 144,00	0,08
gpt2	AdaLoRA (A5)	3,92	50,60	1 770 048,00	1,40
gpt2	(IA) ³ (A6)	3,41	30,29	55 296,00	0,04
Modelo: gpt2-ln-free (Baseline PPL: 36.54)					
ln-free	LoRA (A3)	3,35	28,46	1 179 648,00	0,94
ln-free	BitFit (A4)	3,42	30,48	102 144,00	0,08
ln-free	AdaLoRA (A5)	4,09	59,87	1 770 048,00	1,40
ln-free	(IA) ³ (A6)	3,54	34,54	55 296,00	0,04

Fonte: Elaboração do autor (2025).

4.2 Sweep de inferência

Para caracterizar o impacto prático da simplificação arquitetural, a Tabela 4 compara a taxa de geração do GPT-2 oficial (*baseline*) e da variante LN-Free (substituição por identidades), em diferentes combinações de sequência e *batch*.

Tabela 4 – *Benchmark* de *throughput* de inferência (tokens/s) dos modelos *baseline*, variando o tamanho da sequência (Seq) e o tamanho do lote (Batch).

Modelo	Seq	Batch	Tokens / seg	VRAM (GB)
baseline	256,00	1,00	85,07	0,28
baseline	256,00	8,00	688,79	0,42
baseline	512,00	1,00	85,83	0,30
baseline	512,00	8,00	678,54	0,58
baseline	896,00	1,00	84,56	0,33
baseline	896,00	8,00	665,16	0,82
lnfree_identity	256,00	1,00	104,96	0,53
lnfree_identity	256,00	8,00	829,21	0,66
lnfree_identity	512,00	1,00	102,71	0,55
lnfree_identity	512,00	8,00	821,68	0,82
lnfree_identity	896,00	1,00	104,47	0,58
lnfree_identity	896,00	8,00	816,13	1,05

Fonte: Elaboração do autor (2025).

4.3 Análise dos resultados

A análise dos resultados unificados permite uma comparação direta e robusta entre os modelos e métodos. Os dados das Tabelas 2, 3 e 4 revelam quatro achados principais: (i) o *trade-off* entre os *baselines* A1 e A2, em que o GPT-2 oficial (A1) é significativamente mais preciso (PPL 30,88) do que a variante LN-Free (A2) (PPL 36,54), enquanto o LN-Free com identidades substitutas (A2b) se mostra consistentemente mais rápido na inferência, alcançando ganho de *throughput* de 23,5% no cenário de *batch* unitário e sequência longa (104,47 vs. 84,56 toks/s); (ii) a eficácia superior de LoRA (A3) e BitFit (A4), uma vez que o *fine-tuning* com esses métodos supera a qualidade do *baseline* oficial (PPL 30,88) em ambos os modelos, com LoRA atingindo a melhor qualidade geral do estudo (PPL 26,64) no GPT-2 *vanilla* e excelente desempenho no LN-Free (PPL 28,46), enquanto BitFit apresenta PPL 28,18 (GPT-2) e 30,48 (LN-Free), configurando-se como alternativa

robusta e de baixo custo paramétrico (apenas 0,08% dos parâmetros); (iii) a divergência do AdaLoRA (A5), que não convergiu sob os hiperparâmetros unificados (300 passos, taxa de aprendizado de 2×10^{-4}), resultando em aumento drástico da perplexidade (PPL 50,60 e 59,87) em ambos os modelos, o que reforça a hipótese da Seção 1.4 de que métodos adaptativos mais complexos demandam horizontes de treino mais longos ou taxas de aprendizado mais conservadoras; e (iv) o impacto relativamente modesto do IA³ (A6), método com a menor fração de parâmetros treináveis (0,044%), que melhora o *baseline* LN-Free (de 36,54 para 34,54), mas não supera a qualidade do GPT-2 *vanilla* (de 30,88 para 30,29), sugerindo que, embora altamente eficiente em termos de parâmetros, pode não dispor de capacidade suficiente para otimizar plenamente o modelo sob o orçamento de treino curto adotado.

Do ponto de vista da eficiência de inferência (Tabela 4), a variante LN-Free (A2b) demonstra ganhos de *throughput* consistentes em todas as configurações, variando de aproximadamente +19,7% (batch=1, seq=512) a +23,5% (batch=1, seq=896) em relação ao *baseline*. Esse comportamento é coerente com a hipótese de que a remoção de normalizações reduz operações ponto a ponto e movimentação de memória na etapa de inferência, beneficiando especialmente cenários com sequências longas e baixa concorrência.

5 Discussão

Este capítulo interpreta criticamente os resultados apresentados no Capítulo 4, conectando-os aos objetivos e à hipótese formulados na Introdução. São discutidos os efeitos da remoção de normalização sobre qualidade e eficiência de inferência, o papel das técnicas de adaptação parametricamente eficiente, os compromissos entre fração de parâmetros treináveis e perplexidade, além de limitações e implicações práticas.

5.1 Visão geral e aderência à hipótese

A hipótese de que a remoção de normalização poderia simplificar o grafo de inferência e, por conseguinte, aumentar a taxa de geração foi corroborada pelos experimentos. A variante sem normalização (A2b, LN-Free) apresentou ganhos de *throughput* de até 23,5% em sequência longa e *batch* unitário (Tabela 4), ao passo que a qualidade (perplexidade) piorou quando medida sem adaptação (PPL 36,54 vs. 30,88, Tabela 2).

Por outro lado, a hipótese de recuperação de qualidade via PEFT foi confirmada pela maioria dos métodos. Ao aplicar métodos de adaptação sobre a base LN-Free, observou-se recuperação substancial de qualidade (Tabela 3). Com destaque, o *LoRA* (A3) aplicado ao GPT-2 *vanilla* atingiu a perplexidade de 26,64 em 300 passos, superando significativamente o *baseline* oficial (30,88). Esses achados alinham-se ao objetivo de explorar abordagens sem normalização para inferência eficiente, compensando a queda inicial de qualidade por meio de ajustes parametricamente econômicos.

5.2 Qualidade preditiva: efeitos e recuperação

A remoção da *LayerNorm* (Tabela 2) elevou a perplexidade do modelo de 30,88 para 36,54 na condição bruta (A2), o que era esperado, pois a normalização atua como estabilizador das ativações internas durante o pré-treinamento. A aplicação dos métodos *PEFT* (Tabela 3) revelou diferenças claras na capacidade de recuperação de qualidade. O *LoRA* (A3) apresentou a melhor recuperação em ambos os cenários, atingindo PPL 26,64 no GPT-2 e 28,46 no LN-Free, superando inclusive o *baseline* original com um acréscimo moderado de parâmetros treináveis. O *BitFit* (A4) foi o segundo método mais eficaz, alcançando PPL 28,18 no GPT-2 e 30,48 no LN-Free, também superando o *baseline* em ambos os casos, porém com um custo paramétrico irrisório, dado que apenas os vieses são atualizados.

O IA³ (A6) exerceu impacto mais modesto: melhorou a perplexidade do LN-Free,

reduzindo-a de 36,54 para 34,54, mas não foi capaz de superar de forma consistente o GPT-2 *vanilla*, que passou de PPL 30,88 para 30,29 sob o mesmo orçamento de treino. Já o *AdaLoRA* (A5) falhou em convergir nas condições unificadas adotadas (300 passos, taxa de aprendizado de 2×10^{-4}), resultando em perplexidades superiores a 50,0, o que indica forte sensibilidade a hiperparâmetros e sugere a necessidade de horizontes de treino mais longos ou taxas de aprendizado mais conservadoras do que as testadas.

Em termos de aderência ao orçamento computacional, os resultados indicam que horizontes curtos de treino (300 passos) são suficientes para que *LoRA* e *BitFit* revertam completamente a degradação introduzida pela remoção de normalização, restaurando ou mesmo melhorando a qualidade preditiva sob custo de ajuste relativamente baixo.

5.3 Eficiência de inferência e perfil de ganhos

O *sweep* de inferência (Tabela 4) evidenciou ganhos de taxa de geração para a variante LN-Free (A2b) em todos os cenários. O destaque foi a configuração de sequência longa (896) e *batch* unitário (1), onde o aumento relativo de *throughput* foi mais pronunciado (+23,5%, de 84,56 para 104,47 toks/s).

Esse comportamento é consistente com a redução de operações ponto a ponto e de movimentação de dados associadas à normalização em cada bloco do *Transformer*. Em aplicações interativas ou latência-sensíveis, nas quais *batching* amplo nem sempre é viável, tal perfil de ganhos torna-se especialmente relevante. Em contrapartida, para *batches* maiores (e.g., *batch*=8), a diferença percentual tende a diminuir, mantendo-se na faixa de aproximadamente +20% a +23%, uma vez que o custo de atenção e projeções matriciais passa a dominar a execução, e a remoção de normalização contribui menos para o tempo total por *token*.

5.4 Custo/benefício paramétrico nos métodos de adaptação

Os métodos de adaptação diferem significativamente tanto na fração de parâmetros treináveis quanto na qualidade alcançada (Tabela 3). O *LoRA* (A3) utiliza cerca de 0,94% dos parâmetros como treináveis e, ainda assim, atinge a melhor perplexidade em ambos os modelos (26,64 no GPT-2 e 28,46 no LN-Free). Esse desempenho decorre do poder de ajuste concentrado em projeções críticas de atenção e de camadas *feed-forward*, aliado à possibilidade de *merge* das deltas aos pesos base, o que evita sobrecusto na etapa de inferência. O *BitFit* (A4), por sua vez, trabalha com aproximadamente 0,082% de parâmetros treináveis e ainda assim apresenta a segunda melhor perplexidade (28,18 e 30,48), superando o *baseline* oficial em ambos os casos e evidenciando uma relação custo-benefício excepcional. Já o IA³ (A6) opera com cerca de 0,044% de parâmetros

treináveis — o menor custo entre os métodos avaliados — e atinge PPL 30,29 e 34,54; embora seja parametricamente extremo, sua recuperação de qualidade foi visivelmente inferior à do *BitFit*, que treina cerca do dobro de parâmetros, mas ainda representa uma fração mínima do total. Por fim, o *AdaLoRA* (A5) envolve aproximadamente 1,40% de parâmetros treináveis (o maior custo entre os métodos PEFT analisados), porém o treinamento divergiu sob o regime unificado de 300 passos com taxa de aprendizado de 2×10^{-4} , resultando em PPL superiores a 50,0 e mostrando-se inadequado para esse orçamento de treino, apesar do alto custo paramétrico.

No conjunto, os resultados delineiam um espectro de escolhas: quando a prioridade é maximizar a qualidade final, o *LoRA* desponta como opção preferencial; quando o requisito dominante é a eficiência paramétrica, o *BitFit* oferece um balanço superior ao IA³, alcançando perplexidade substancialmente melhor com um custo ainda irrisório (0,082% dos parâmetros).

5.5 Interpretação prática e escolhas de projeto

Do ponto de vista de engenharia, os resultados apontam para um *trade-off* claro entre qualidade máxima e eficiência máxima, que pode ser organizado em três cenários principais. Em primeiro lugar, para quem busca *pico de qualidade*, a combinação *gpt2 + LoRA* (A3) é a opção vencedora: ela atinge a menor perplexidade do estudo (PPL 26,64), superando confortavelmente o *baseline* original, ao mesmo tempo em que preserva a velocidade de inferência observada para o GPT-2 oficial (Tabela 4). Em segundo lugar, para um balanço prático entre velocidade e qualidade, a combinação *ln-free + LoRA* (A3) se mostra particularmente atraente: embora sua PPL (28,46) seja ligeiramente superior à do campeão de qualidade, ainda melhora o *baseline* original (30,88) e se beneficia da arquitetura LN-Free, que oferece ganhos de *throughput* da ordem de 20–23% na inferência. Por fim, para um cenário de *baixo custo* de ajuste, a combinação *ln-free + BitFit* (A4) desponta como escolha robusta, alcançando PPL 30,48 (melhor que o *baseline* oficial), herdando os ganhos de velocidade do LN-Free e mantendo um custo de treino praticamente nulo, dado que apenas 0,082% dos parâmetros são atualizados.

Quando a restrição principal é manter o número de parâmetros treináveis no nível mais baixo possível, o IA³ oferece uma alternativa ainda mais parcimoniosa; entretanto, dentro do orçamento de treino adotado, o *BitFit* apresentou retorno superior, conciliando melhor qualidade com custo ainda bastante reduzido. Já no caso do *AdaLoRA*, os resultados indicam que o método não se comporta como uma solução *drop-in*: sua divergência sob 300 passos e taxa de aprendizado de 2×10^{-4} sugere a necessidade de sintonização cuidadosa de hiperparâmetros (taxa de aprendizado e número de passos), o que o torna inadequado para o regime de treino curto explorado neste estudo.

5.6 Ameaças à validade e limitações

Algumas fontes de viés e limitações metodológicas merecem atenção. Em primeiro lugar, quanto a dados e tarefa, os resultados de perplexidade foram obtidos em um único conjunto (*wikitext-2-raw-v1*) e sob uma métrica única; não foram avaliadas tarefas complementares nem o *generalization gap* em domínios distintos, o que restringe o alcance das conclusões. Em segundo lugar, o horizonte de treino adotado (300 passos) foi propositalmente contido, de modo a refletir um cenário de orçamento curto; os resultados do *AdaLoRA* (A5), que divergiu, indicam que esse horizonte foi insuficiente para métodos que requerem agendamentos mais longos, e é razoável supor que outros métodos, como o IA³, também pudessem apresentar ganhos adicionais com mais passos. Em terceiro lugar, as medições de *runtime* em ambiente compartilhado estão sujeitas a flutuações; buscou-se mitigar esse efeito por meio de aquecimento, sincronização explícita do dispositivo e repetição de medições, mas pequenas variações nos valores de *tokens por segundo* são esperadas. Por fim, o escopo do modelo avaliou apenas um único porte (GPT-2 *small*); qualquer extrapolação para modelos maiores ou arquiteturas mais recentes deve ser confirmada empiricamente, uma vez que o balanço entre custo de normalização, largura do modelo e profundidade pode se alterar em outras escalas.

5.7 Implicações e recomendações

Do ponto de vista prático, os achados permitem derivar algumas recomendações de projeto. Para cenários em que o perfil de uso enfatiza sequências longas, *batch* reduzido e latência por requisição crítica, recomenda-se empregar a variante LN-Free como ponto de partida, explorando seus ganhos de *throughput* evidenciados na Tabela 4. Quando o objetivo é maximizar a recuperação de qualidade sob orçamentos de treino curtos, o *LoRA* (A3) se apresenta como primeira escolha, devendo-se realizar o *merge* das deltas aos pesos base antes da implantação, conforme indicado na Tabela 3. Em cenários em que a prioridade é um balanço entre baixo custo paramétrico e boa qualidade, o *BitFit* (A4) mostra-se particularmente adequado, superando o IA³ (A6) nessa relação ao atingir perplexidades menores com uma fração ainda reduzida de parâmetros treináveis. Já o *AdaLoRA* (A5) deve ser tratado com cautela em regimes de treino curtos: a divergência observada sob 300 passos e taxa de aprendizado de 2×10^{-4} indica que o método não se comporta como solução *drop-in* e requer sintonização mais cuidadosa de hiperparâmetros (taxa de aprendizado e número de passos), não sendo recomendado, tal como configurado neste estudo, para cenários em que o orçamento de treino é estritamente limitado.

6 Conclusão

Este capítulo sintetiza as respostas às questões de pesquisa, confrontando-as com os objetivos estabelecidos na Introdução, e apresenta recomendações práticas, limitações e direções para trabalhos futuros. Mantém-se o foco na avaliação da remoção de *LayerNorm* (LN-Free) em um GPT-2 *small* e no uso de métodos de adaptação parametricamente eficiente para recuperar qualidade preditiva, sem comprometer os ganhos de eficiência de inferência.

Os resultados obtidos, baseados em um experimento controlado com parâmetros unificados (Tabelas 2, 3 e 4), permitem enunciar algumas conclusões principais. Em primeiro lugar, no que diz respeito à eficiência de inferência com LN-Free, a simplificação arquitetural por remoção de *LayerNorm* (A2b) aumentou o *throughput* de geração em todas as configurações avaliadas. Conforme a Tabela 4, os ganhos foram mais pronunciados em sequência longa e *batch* unitário, com aumento de $84,56 \rightarrow 104,47 \text{ tok/s}$ (+23,5%) em $\text{seq} = 896$, $\text{batch} = 1$. Em segundo lugar, quanto ao impacto em qualidade, a variante LN-Free (A2), sem qualquer adaptação, apresentou perplexidade maior (PPL 36,54) que o GPT-2 *small* oficial (A1, PPL 30,88), confirmando a expectativa de decréscimo de qualidade após remover a normalização de um modelo pré-treinado (Tabela 2). Em terceiro lugar, a recuperação de desempenho com *PEFT* foi heterogênea: os métodos de adaptação (Tabela 3) apresentaram eficácia variada, com *LoRA* (A3) despontando como o método mais poderoso, ao atingir PPL 26,64 no GPT-2 e 28,46 no LN-Free, superando o *baseline* oficial em ambos os casos, e *BitFit* (A4) ocupando a segunda colocação (PPL 28,18 e 30,48), também superando o *baseline* em ambos os modelos. O IA^3 (A6) teve um impacto mais modesto, melhorando o LN-Free (PPL 34,54), mas sem conseguir melhorar de forma substantiva o GPT-2 *vanilla* (PPL 30,29). Em quarto lugar, o *AdaLoRA* (A5) falhou em convergir sob os hiperparâmetros unificados (300 passos, taxa de aprendizado de 2×10^{-4}), divergindo para PPLs superiores a 50,0, o que indica inadequação ao regime de treino curto adotado ou alta sensibilidade à taxa de aprendizado. Por fim, a análise de custo/benefício paramétrico revela um *trade-off* interessante: o *LoRA*, com cerca de 0,94% de parâmetros treináveis, atingiu a melhor qualidade absoluta, enquanto o *BitFit*, com apenas 0,082% de parâmetros, ofereceu o melhor balanço entre economia paramétrica e perplexidade, superando significativamente o IA^3 (0,044%) em qualidade, apesar de seu custo ainda irrisório. Nesse contexto, a combinação **ln-free + LoRA** (PPL 28,46) emergiu como a mais equilibrada, por resultar em um modelo simultaneamente mais rápido (devido à arquitetura LN-Free) e mais preciso que o *baseline* original (PPL 30,88).

À luz desses achados, é possível responder aos objetivos específicos do trabalho. A comparação entre o *baseline* oficial e o LN-Free confirmou, em primeiro lugar, a melhora

de *throughput* com LN-Free (Tabela 4) e, em paralelo, a piora inicial de perplexidade (36,54 vs. 30,88) na ausência de adaptação (Tabela 2). Em relação à mensuração de ganhos por sequência e tamanho de *batch*, observou-se que os ganhos relativos de *throughput* do LN-Free foram maiores em comprimentos de sequência longos e *batch* = 1 (com +23,5% em *seq* = 896), enquanto para *batches* maiores a diferença percentual diminuiu, embora permaneça significativa ao longo das configurações avaliadas (Tabela 4). No que se refere à avaliação dos métodos *PEFT* sobre LN-Free, constatou-se que *LoRA* (PPL 28,46) e *BitFit* (PPL 30,48) foram capazes de recuperar e ultrapassar a qualidade do *baseline* oficial (PPL 30,88), ao passo que o IA³ (PPL 34,54) melhorou o *baseline* LN-Free (PPL 36,54), mas não alcançou o modelo oficial. O *AdaLoRA* (PPL 59,87) falhou em convergir neste cenário, reiterando sua inadequação ao orçamento de treino adotado (Tabela 3). Por fim, a relação entre fração treinável, perplexidade e custo de inferência mostrou que *LoRA* (0,94%) e *BitFit* (0,082%) apresentam o melhor custo-benefício, enquanto o IA³ (0,044%) foi menos eficaz que o *BitFit*, sugerindo que a economia extrema de parâmetros, neste experimento, teve como contrapartida uma perda de qualidade. Métodos com fusão de deltas, como o *LoRA*, não adicionam custo adicional na geração após o *merge*, o que os torna particularmente atrativos quando combinados com a arquitetura LN-Free.

Do ponto de vista prático, os resultados permitem derivar recomendações de projeto. Para cenários de baixa simultaneidade e sequências longas, a adoção de LN-Free é vantajosa no caminho de inferência, pois reduz a latência média por *token* (Tabela 4). Quando o objetivo é maximizar a qualidade sob orçamentos curtos de *fine-tuning*, o *LoRA* se mostra a primeira escolha, e a realização do *merge* das deltas antes da implantação preserva os ganhos de eficiência da variante LN-Free (Tabela 3). Em contextos em que a prioridade é minimizar o número de parâmetros treináveis, o *BitFit* (0,082%) oferece um balanço consideravelmente superior ao IA³ (0,044%), alcançando uma qualidade significativamente melhor com um custo ainda muito baixo. Já o *AdaLoRA* demonstrou instabilidade e não deve ser utilizado como solução *drop-in* em regimes de treino curtos, exigindo sintonização cuidadosa de taxa de aprendizado e número de passos para se tornar competitivo.

Apesar dos cuidados metodológicos, o estudo apresenta limitações relevantes. A avaliação de qualidade foi restrita à perplexidade sobre o conjunto *wikitext-2-raw-v1*, sem incluir métricas extrínsecas de tarefa ou análises em domínios distintos, o que limita a generalização dos resultados a outros cenários de uso. O horizonte de treino foi propositalmente curto (300 passos), o que se mostrou insuficiente para métodos como o *AdaLoRA*, que divergiu, e pode ter limitado o potencial de métodos mais sensíveis à dinâmica de otimização, como o IA³. Além disso, as medições de *runtime* em ambiente compartilhado, ainda que mitigadas por aquecimento, sincronização explícita e repetição de execuções, permanecem sujeitas a flutuações na métrica de *tokens por segundo*. Por fim, o escopo do estudo foi restrito a um único porte de modelo (GPT-2 *small*); extrapolações para modelos maiores ou arquiteturas mais modernas requerem confirmação empírica, dado que

a relação entre custo de normalização, profundidade e largura do modelo pode se alterar em outras escalas.

Essas limitações abrem espaço para trabalhos futuros. Uma direção natural consiste em expandir a avaliação para tarefas extrínsecas e métricas orientadas ao uso final, incluindo não apenas perplexidade, mas também indicadores de qualidade em tarefas downstream, além de métricas de serviço, como latências p95/p99 e o impacto de estratégias de *batching* dinâmico. Outra linha promissora é investigar variantes de normalização alternativas, como a *RMSNorm*, e esquemas híbridos de remoção parcial, comparando-os diretamente com a abordagem LN-Free. Adicionalmente, a exploração de alvos e *ranks* diferenciados por camada em *LoRA* e *AdaLoRA*, bem como a sensibilidade do IA³ à escolha de subespaços de ativações, pode esclarecer melhor o espaço de configurações eficientes. Por fim, repetir o estudo em modelos de maior porte permitiria avaliar como os ganhos e custos escalam com a dimensionalidade d , o número de cabeças de atenção e a profundidade da arquitetura, aproximando a análise de cenários mais alinhados a LLMs de produção.

Em síntese, a evidência experimental sustenta que a remoção de *LayerNorm* pode tornar a inferência mais eficiente em modelos de linguagem, com ganhos de *throughput* de até aproximadamente 23,5%, e que a aplicação de adaptação parametricamente eficiente — em especial *LoRA* e *BitFit* — é capaz de recuperar e, em alguns casos, superar a qualidade do *baseline*, mantendo o caminho de inferência enxuto. O estudo revelou um *trade-off* claro: o pico de qualidade preditiva foi atingido pelo modelo *gpt2* + *LoRA* (PPL 26,64), mas a combinação *ln-free* + *LoRA* (PPL 28,46) apresentou o balanço de engenharia mais promissor, resultando em um modelo que é, ao mesmo tempo, mais rápido e significativamente mais preciso do que o *baseline* oficial (PPL 30,88). Assim, a combinação de simplificação arquitetural e *fine-tuning* leve configura uma estratégia concreta para reduzir latência e custo operacional sem comprometer de forma material a qualidade preditiva, alinhando-se ao objetivo de explorar abordagens sem normalização para inferência eficiente em grandes modelos de linguagem.

Referências Bibliográficas

BA, J. L.; KIROS, J. R.; HINTON, G. E. Layer normalization. *arXiv*, 2016. Disponível em: <<https://arxiv.org/abs/1607.06450>>. Acesso em: 28 out. 2025.

BARONI, L.; KHARA, G.; SCHAEFFER, J.; SUBKHANKULOV, M.; HEIMERSHEIM, S. Transformers don't need layernorm at inference time: Scaling layernorm removal to gpt-2 xl and the implications for mechanistic interpretability. *arXiv*, 2025. Disponível em: <<https://arxiv.org/abs/2507.02559>>. Acesso em: 28 out. 2025.

CANDIDO, D. G. *lnfree-gpt2-peft-tcc: código e resultados dos experimentos do TCC*. 2025. Repositório GitHub. Disponível em: <<https://github.com/danielgcandido/lnfree-gpt2-peft-tcc>>. Acesso em: 28 out. 2025.

CHITTY-VENKATA, K. T.; MITTAL, S.; EMANI, M.; VISHWANATH, V.; SOMANI, A. K. A survey of techniques for optimizing transformer inference. *Journal of Systems Architecture*, v. 144, p. 102990, 2023. Disponível em: <<https://doi.org/10.1016/j.sysarc.2023.102990>>. Acesso em: 28 out. 2025.

DAO, T.; FU, D. Y.; ERMON, S.; RUDRA, A.; RÉ, C. Flashattention: fast and memory-efficient exact attention with IO-awareness. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2022. Disponível em: <<https://arxiv.org/abs/2205.14135>>. Acesso em: 28 out. 2025.

HEIMERSHEIM, S. You can remove GPT-2's LayerNorm by fine-tuning. *arXiv*, 2024. V2, 19 nov. 2024. Disponível em: <<https://arxiv.org/abs/2409.13710>>. Acesso em: 28 out. 2025.

HU, E. J.; SHEN, Y.; WALLIS, P.; ALLEN-ZHU, Z.; LI, Y.; WANG, S.; WANG, L.; CHEN, W. Lora: low-rank adaptation of large language models. *arXiv*, 2021. Disponível em: <<https://arxiv.org/abs/2106.09685>>. Acesso em: 28 out. 2025.

Intel Developer Zone. *Optimize Transformer model inference on Intel® processors*. 2025. Disponível em: <<https://www.intel.com/content/us/en/developer/articles/technical/optimize-transformer-model-inference-processors.html>>. Acesso em: 28 out. 2025.

JIANG, Z.; GU, J.; ZHU, H.; PAN, D. Z. Pre-rmsnorm and pre-crmsnorm transformers: Equivalent and efficient pre-ln transformers. In: *Advances in Neural Information Processing Systems 36: Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS 2023)*. New Orleans, United States: Neural Information Processing Systems Foundation, 2023. (Advances in Neural Information Processing Systems, v. 36). Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/file/8f1bacee31caf990a4f08d84f0ccb322-Paper-Conference.pdf>. Acesso em: 28 out. 2025.

LabelYourData. *LLM inference: techniques for optimized deployment in 2024*. 2025. Disponível em: <<https://labelyourdata.com/articles/llm-inference>>. Acesso em: 28 out. 2025.

LIU, H. Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning. *arXiv*, 2022. Disponível em: <<https://arxiv.org/abs/2205.05638>>. Acesso em: 28 out. 2025.

RAIAAN, M. A. K.; MUKTA, M. S. H.; FATEMA, K.; FAHAD, N. M.; SAKIB, S.; MIM, M. M. J.; AHMAD, J.; ALI, M. E.; AZAM, S. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE Access*, v. 12, p. 26839–26874, 2024. Disponível em: <<https://doi.org/10.1109/ACCESS.2024.3365742>>.

SHAO, M.; BASIT, A.; KARRI, R.; SHAFIQUE, M. Survey of different large language model architectures: Trends, benchmarks, and challenges. *IEEE Access*, v. 12, p. 188664–188706, 2024. Disponível em: <<https://doi.org/10.1109/ACCESS.2024.3482107>>.

SONG, Y.; MI, Z.; XIE, H.; CHEN, H. Powerinfer: fast large language model serving with a consumer-grade GPU. In: *Proceedings of the ACM Symposium on Operating Systems Principles (SOSP)*. [s.n.], 2024. Disponível em: <<https://arxiv.org/abs/2312.12456>>. Acesso em: 28 out. 2025.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, Ł.; POLOSUKHIN, I. Attention is all you need. In: *Advances in Neural Information Processing Systems (NeurIPS)*. [s.n.], 2017. Disponível em: <<https://papers.nips.cc/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>>. Acesso em: 28 out. 2025.

XIONG, R.; YANG, Y.; HE, D.; ZHENG, K.; ZHENG, S.; XING, C.; ZHANG, H.; LAN, Y.; WANG, L.; LIU, T.-Y. On layer normalization in the transformer architecture. In: *Proceedings of the 37th International Conference on Machine Learning (ICML)*. [s.n.], 2020. Disponível em: <<https://arxiv.org/abs/2002.04745>>. Acesso em: 28 out. 2025.

ZAKEN, E. B.; RAVFOGEL, S.; GOLDBERG, Y. Bitfit: simple parameter-efficient fine-tuning for transformer-based masked language-models. In: *Findings of the Association for Computational Linguistics (ACL 2022)*. [s.n.], 2022. Disponível em: <<https://arxiv.org/abs/2106.10199>>. Acesso em: 28 out. 2025.

ZHANG, B.; SENNRICH, R. Root mean square layer normalization. *arXiv*, 2019. Disponível em: <<https://arxiv.org/abs/1910.07467>>. Acesso em: 28 out. 2025.

ZHANG, R. Adaptive budget allocation in parameter-efficient fine-tuning via AdaLoRA. *arXiv*, 2023. Disponível em: <<https://arxiv.org/abs/2303.10512>>. Acesso em: 28 out. 2025.

ZHANG, R.; JIANG, H.; WANG, W.; LIU, J. Optimization methods, challenges, and opportunities for edge inference: A comprehensive survey. *Electronics*, v. 14, n. 7, p. 1345, 2025. Disponível em: <<https://doi.org/10.3390/electronics14071345>>.

ZHUO, Z.; ZENG, Y.; WANG, Y.; ZHANG, S.; YANG, J.; LI, X.; ZHOU, X.; MA, J. Hybridnorm: Towards stable and efficient transformer training via hybrid normalization. *arXiv*, 2025. Disponível em: <<https://arxiv.org/abs/2503.04598>>. Acesso em: 28 out. 2025.